Applied Reinforcement Learning

Generalizing with Neural Networks

Riccardo Berta

2026.10.07

Contents

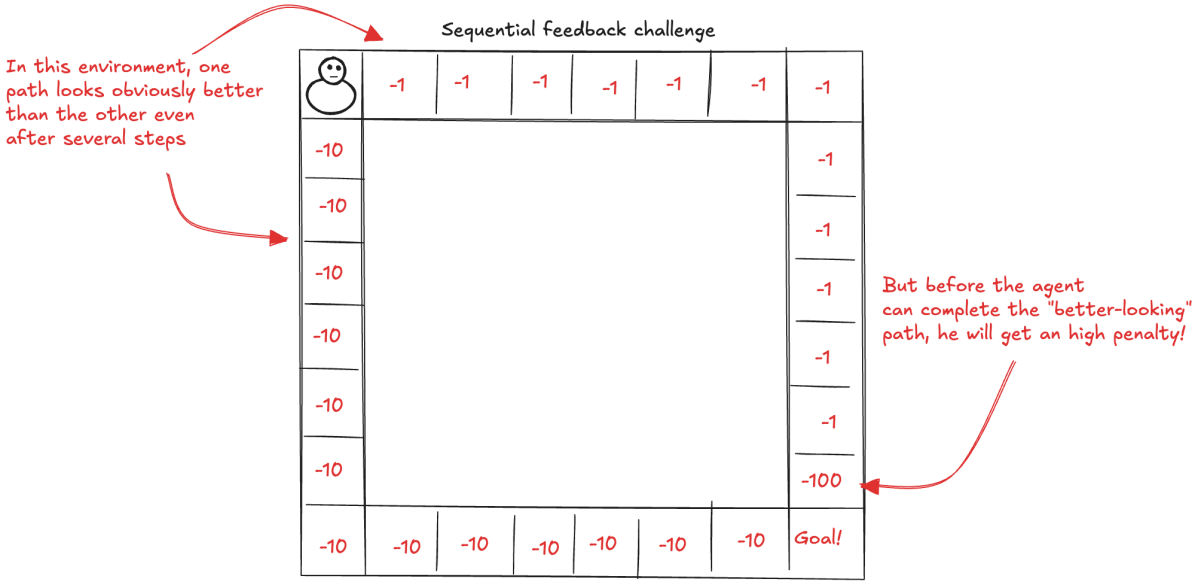
1 Generalizing with Neural Networks **1**
1.1 The Need for Generalization . . . 4
1.1.1 From Tables to Parameters . . . 6
1.1.2 The Cart-Pole Environment . . . 8
1.2 Neural Fitted Q (NFQ) . . . 12
1.2.1 Network Architecture . . . 13
1.2.2 Universal Approximation . . . 14
1.2.3 Loss Function . . . 16
1.2.4 Optimization . . . 18
1.2.5 Dataset and training . . . 20
1.2.6 Algorithm . . . 23
1.2.7 Solve Cart-Pole . . . 24
1.3 Deep Q-Networks (DQN) . . . 30
1.3.1 Deadly Triad . . . 31
1.3.2 Experience Replay . . . 32
1.3.3 Target Networks . . . 35
1.3.4 Exploration Strategies . . . 37
1.3.5 Algorithm . . . 38
1.3.6 Solve Cart-Pole . . . 40
1.3.7 Double Deep Q-Learning (DDQN) . . . 44
1.3.8 Prioritized Experience Replay (PER) . . . 48
1.3.9 Dueling Architecture . . . 55
1.4 Limitations and Outlook . . . 59

1 Generalizing with Neural Networks

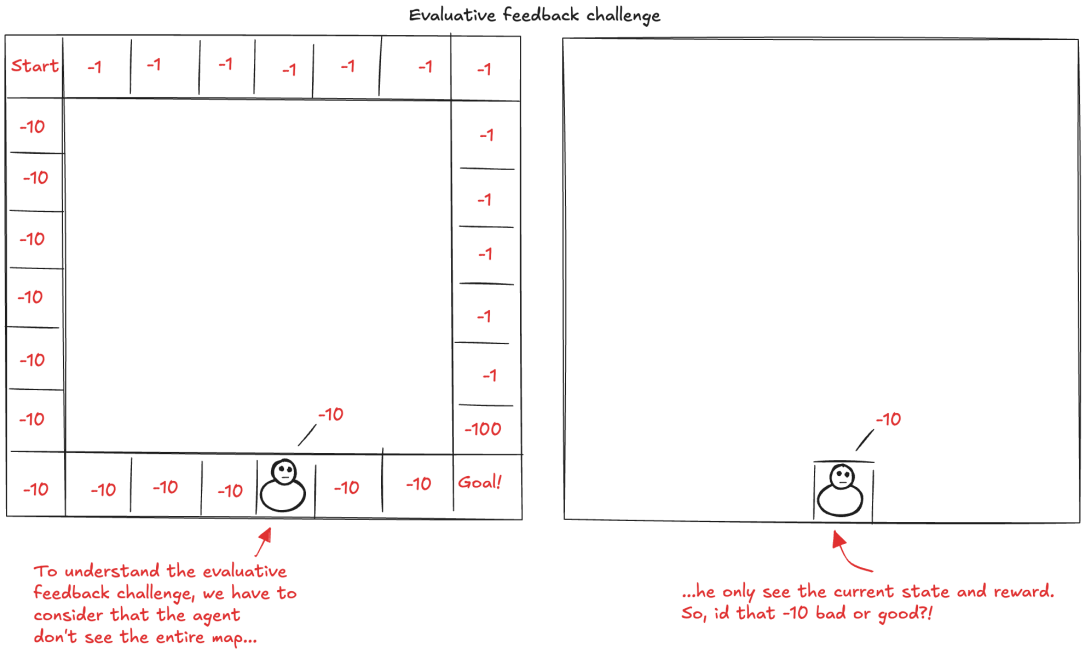
Tabular reinforcement learning represents value functions with one entry for each state or state-action pair. This is effective in small environments, but it quickly becomes impractical when the state space is large, continuous, or high-dimensional. In these settings, the agent must do more than memorize values: it must **generalize** from the states it has experienced to states it has never seen before. Neural networks provide a flexible way to achieve this. Instead of storing values in a table, **we use a neural network to approximate an action-value function**. The network receives a representation of the state, or of a state-action pair, and outputs value estimates that guide the agent's decisions. The motivation for all of this comes from the nature of reinforcement learning

itself. In reinforcement learning, agents learn from feedback that is simultaneously **evaluative, delayed, and sampled**.

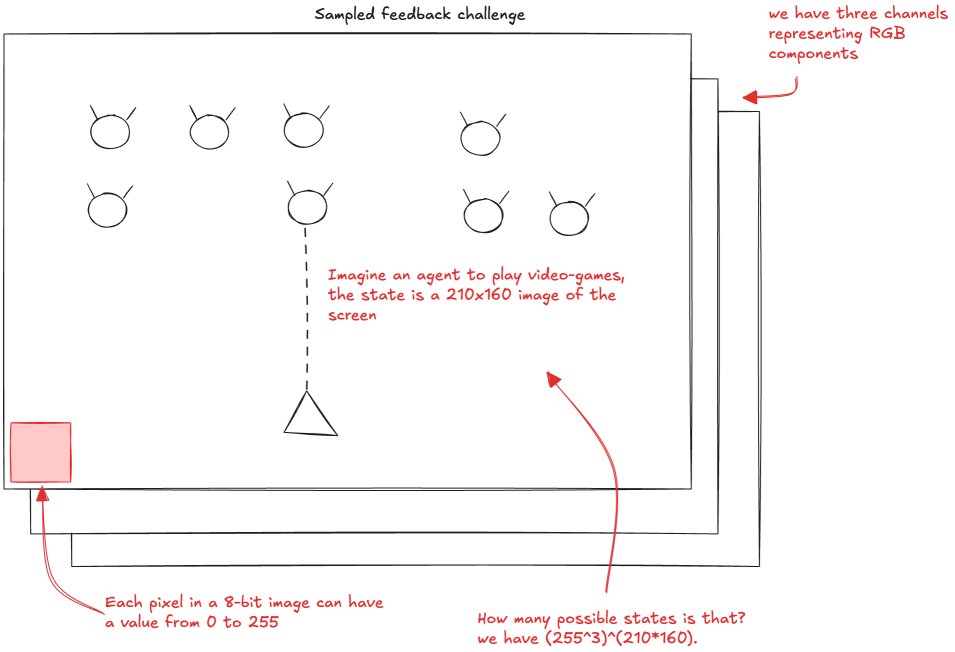
Delayed feedback: imagine a chess game in which you make several poor moves early on, but their consequences become apparent only at the end of the game. This kind of feedback makes it difficult to identify where the final outcome actually comes from, and gives rise to the **temporal credit assignment problem**: the challenge of determining which state, action, or state-action pair is responsible for a given reward. Its opposite is **immediate feedback**. In supervised learning, decisions do not have long-term consequences. For example, in an image classification task, classifying an image correctly or incorrectly does not affect the model's future performance.



Evaluative feedback: feedback in reinforcement learning is only relative, because the environment is uncertain and the agent does not know its true dynamics. In particular, the agent has no direct access to the transition function or the reward function. As a result, it must learn by interacting with the environment and exploring different actions. The difficulty is that exploration comes at a cost: while the agent is trying new actions to gain information, it may miss opportunities to exploit what it already knows. This tension is known as the **exploration-exploitation trade-off**. The opposite is **supervised feedback**. In a classification problem, for example, the model is given the correct label for each training sample. It does not need to discover the right answer through trial and error.



Sampled feedback: in real-world problems, agents are unlikely to observe every possible situation or receive feedback for every possible state-action pair. Instead, they must learn from a limited set of sampled experiences and generalize from them in order to make intelligent decisions in new situations. The opposite is exhaustive feedback, where the agent has access to all possible samples. In tabular reinforcement learning, this assumption can be reasonable in small environments with finite state and action spaces. If the agent samples long enough, it can eventually gather the information it needs, and many tabular methods come with convergence guarantees under assumptions such as "every state-action pair is sampled infinitely often". These assumptions may hold in small grid worlds, but they break down in realistic environments. Consider, for example, an agent playing a video game:



The number of possible states in a video game is enormous. This exposes the limitations of traditional tabular reinforcement learning and motivates the use of **learning methods** that can handle high-dimensional state spaces and learn effective policies from limited experience.

1.1 The Need for Generalization

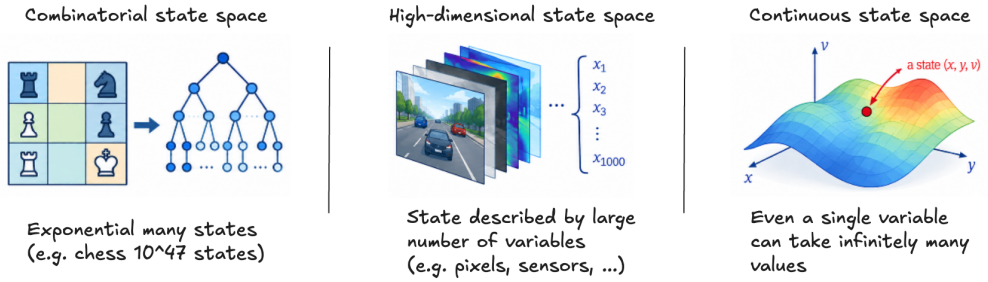
The main limitation of the methods we have seen so far is that they represent value functions using tables. For this reason, they are known as **tabular reinforcement learning** methods. This representation works well in small environments with a limited number of states and actions, but it quickly becomes impractical in complex problems. For example, Q-learning estimates the optimal action-value function using a table, or matrix, indexed by states and actions:

An action-state value function is indexed by the state and the action and it returns the expected reward-to-go for taking that action at that state

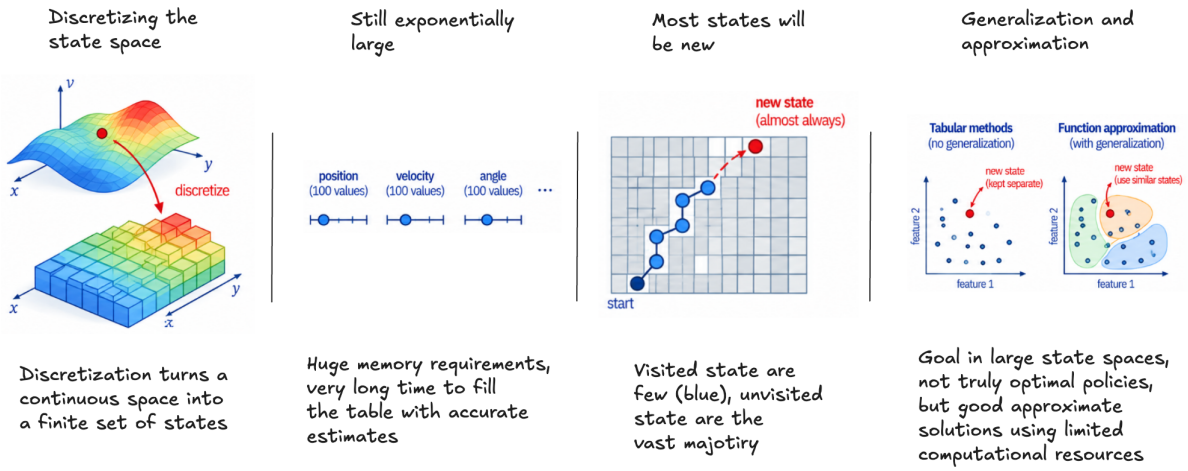
	Q	States 0	1	2	3
Actions	0	-1.5	-0.2	1.2	5.7
	1	4,2	-2.1	2.7	6.1

In many tasks where we would like to apply reinforcement learning, the **state space is combi-**

natorial and enormous. Environments may have **high-dimensional state spaces**, where each state is described by a large number of variables, or they may include **continuous variables**, where even a single variable can take infinitely many values.



This immediately raises a practical question: how could we build a table that accounts for all of these possibilities? One option is to **discretize the state space**, but even then the number of possible values can remain extremely large, making tabular solutions **impractical** because of the size of the required tables. The issue is not only memory. It is also the **time needed to fill those tables accurately**. In many realistic tasks, almost every state the agent encounters will be new. To act sensibly, the agent must therefore **generalize** from previous experience with other states that are, in some relevant sense, similar to the current one. Tabular methods are inefficient precisely because they lack this ability to generalize. They treat states as separate entries, which prevents them from discovering underlying relationships among states. The key question, then, is how experience gathered from a limited subset of the state space can be used to produce a useful approximation over a much larger one. Under these conditions, we should not expect to find truly optimal policies. A more realistic goal is to find **good approximate solutions** using **limited computational resources**.



Generalization from examples has already been studied extensively, so we do not need to invent entirely new methods for reinforcement learning. Instead, we can combine reinforcement

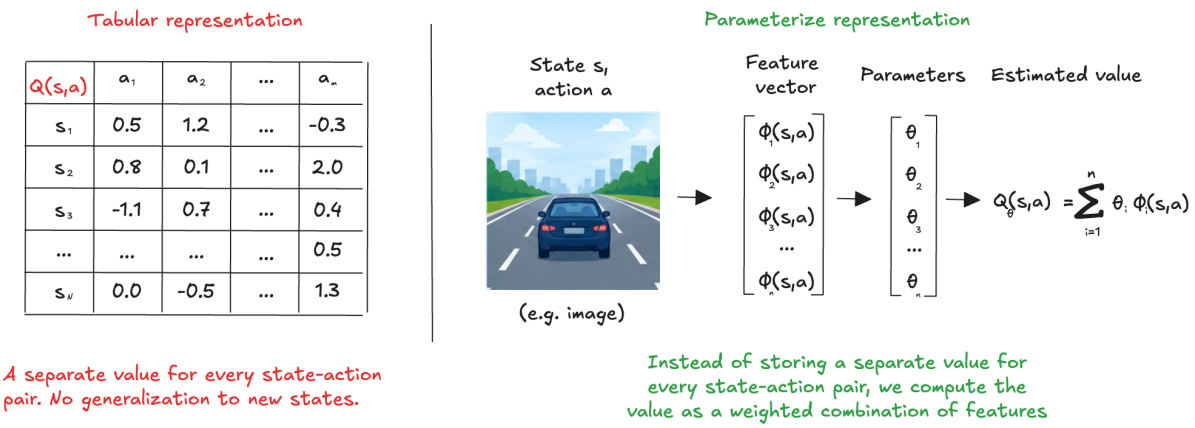
learning algorithms with existing generalization techniques. This kind of generalization is usually called **function approximation**. The idea is to take examples from a target function, such as a value function, and use them to construct an approximation of the full function. In this sense, function approximation is a form of **supervised learning**. In principle, almost any supervised learning method can be used as a function approximator inside a reinforcement learning algorithm, although in practice some methods fit this role more naturally than others.

1.1.1 From Tables to Parameters

The simplest way to move from a table to a representation that can generalize is to describe each state-action pair using a **feature vector**. Instead of storing a separate value for every possible pair, we compute the value as a weighted combination of features:

$\phi(s,a) \in \mathbb{R}^n$

$$Q_\theta(s,a) = \theta^\top \phi(s,a) = \sum_{i=1}^{n} \theta_i \phi_i(s,a)$$



The vector contains the **parameters** of the approximator. From this point on, **learning a value function no longer means filling the cells of a table**. It means **adjusting the parameters of a function approximator**. Linear function approximation is the natural starting point for this idea for two reasons. First, **a table is a special case**. If the feature vector is a one-hot encoding of a state-action pair, then the weighted sum simply selects the corresponding table entry. In other words, tabular reinforcement learning can be seen as linear function approximation with one independent parameter for every state-action pair:

$n = |\mathcal{S}||\mathcal{A}|$

In this case, there is **no generalization**: updating one state-action pair changes only that entry. The real advantage appears when we use **fewer parameters** than state-action pairs:

$n \ll |\mathcal{S}||\mathcal{A}|$

Now, **different pairs may share features**, and therefore share parameters. An update based on one state-action pair can also affect the estimated value of other pairs with similar features. This is exactly the kind of **generalization** we need, although it will also introduce new difficulties. Second, **the gradient of a linear approximator is especially simple**:

$\nabla_\theta Q(s, a; \theta) = \phi(s, a)$

As a result, the familiar incremental update pattern used throughout the course:

NewEstimate ← **OldEstimate + StepSize** × (**Target - OldEstimate**)

can be reinterpreted in terms of function approximation. In tabular methods, the old estimate is a single table entry. Updating it changes only that one entry. With function approximation, however, the estimate is computed from the parameter vector:

$Q(s, a; \theta)$

So we cannot directly replace one table entry. Instead, we adjust the parameters so that the new value predicted for the current state-action pair moves closer to the target. The prediction error is:

$\text{Target} - Q(s, a; \theta)$

For a linear approximator, the gradient tells us how each parameter affects the current prediction. Since the gradient is simply the feature vector:

$\nabla_\theta Q_\theta(s, a) = \phi(s, a)$

we update the parameters in the direction of that feature vector:

$\theta \leftarrow \theta + \alpha[\text{Target} - Q_\theta(s, a)]\phi(s, a)$

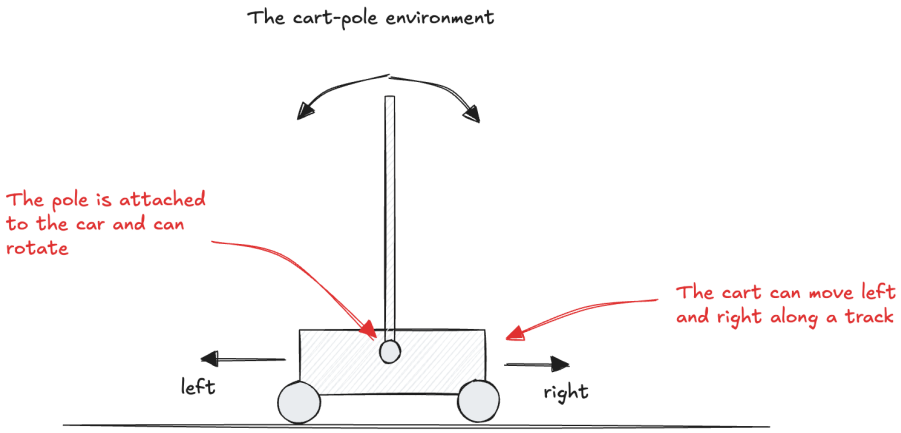
In words, the update says: change the parameters in proportion to the prediction error, scaled by the step size, and distribute that change according to the features that were active for the current state-action pair. A neural network is a **non-linear function approximator**. Its features are no longer designed by hand, they are learned by the hidden layers. The output layer then combines those learned features to produce the final value estimate. The update has the same general structure as before, but the gradient is now computed by backpropagation rather than being equal to the feature vector:

$\nabla_\theta Q_\theta(s, a)$

The price we pay for this additional flexibility is that the convergence guarantees available in the linear case no longer generally apply.

1.1.2 The Cart-Pole Environment

To illustrate why generalization is needed, we will use a classic control problem: **Cart-Pole**. The goal is to balance a pole attached to a cart that moves along a frictionless track. The pole starts upright, and the agent must prevent it from falling by pushing the cart either left or right. Cart-Pole is useful because it is simple enough to train quickly, but still challenging enough to show the limitations of tabular methods. Its state space is low-dimensional, but **continuous**, which means that a table-based representation is already inconvenient and function approximation becomes useful.



The state is a vector with four continuous variables:

- the cart position on the track
- the cart velocity
- the pole angle with respect to the vertical
- the pole angular velocity

In every state, the agent can choose between two actions:

- action 0 pushes the cart to the left
- action 1 pushes the cart to the right

The force has fixed magnitude, equal to 10 N. The agent only chooses the direction of the force. The reward is +1 at every time step, including the final one. Therefore, the return of an episode is simply its length. The episode terminates when the agent fails(the pole angle exceeds a threshold from the vertical), or if the cart position leaves the horizontal range. Since the reward is +1 per step and episodes are truncated after 500 steps, the maximum return is 500. The environment is conventionally considered solved when the average return exceeds 475. The Cart-Pole environment is available in the Gymnasium library:

```
import gymnasium

cart_pole = gymnasium.make('CartPole-v1', render_mode="rgb_array");
```

We can get the state space and the action space sizes of the environment:

```
state_size = cart_pole.observation_space.shape;
action_size = cart_pole.action_space.n;

print('State Space: ', state_size);
print('Action Space: ', action_size);
```

```
State Space:  (4,)
Action Space:  2
```

We want to be able to reproduce the results, so we will fix the random seed:

```
import torch;
import numpy as np;
import random;

# set the seeds for reproducibility of results
seed = 34;

torch.manual_seed(seed);
np.random.seed(seed);
random.seed(seed);

cart_pole.reset(seed=seed)
cart_pole.action_space.seed(seed)
cart_pole.observation_space.seed(seed);
```

As a baseline for comparison, we use a **random policy**, which selects actions uniformly at random from the action space. Although this policy does not learn, it provides a useful reference point: it allows us to measure how much a learned policy improves over uninformed behavior.

```python
def random_pi(state):
    # selects an action uniformly at random
    # from the environment's action space.
    return cart_pole.action_space.sample()
```

To better understand how the agent interacts with the environment, we define a visualization function that shows the policy in action, step by step. This allows us to observe how the agent's decisions unfold over the course of an episode.

```python
import matplotlib.pyplot as plt
from IPython.display import clear_output

def show_policy(env, pi, n_episodes=1, max_steps=500):

    # start the episodes
    for episode in range(n_episodes):

        # reset the environment
        state, _ = env.reset();
        done = False;

        # initialize the step counter
        step = 0;

        # run the episode until done or max_steps
        while not done and step < max_steps:

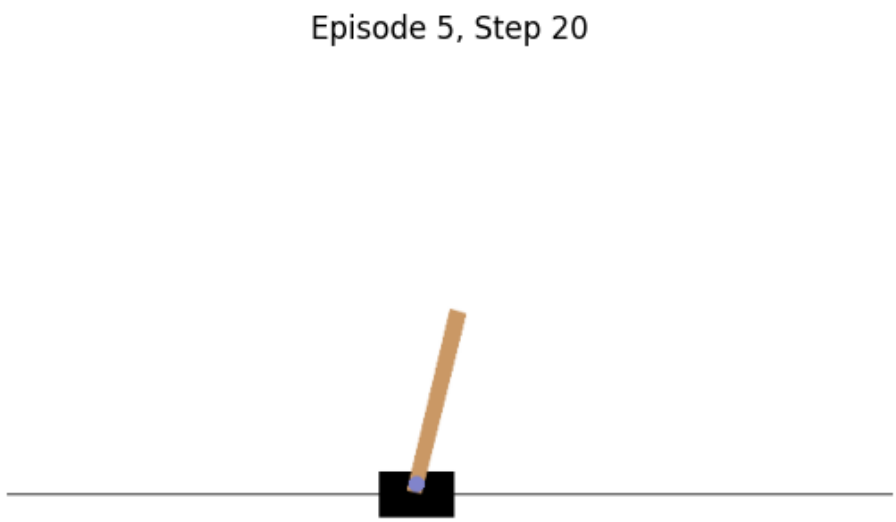
            # make a step
            action = pi(state);
            state, _, terminated, truncated, _ = env.step(action);
            done = terminated or truncated;

            # render the environment
            clear_output(wait=True)
            plt.imshow(env.render())
            plt.axis("off")
            plt.title(f"Episode {episode+1}, Step {step}")
            plt.show()
            plt.close()

            # increment the step counter
            step += 1;
```

When applied to the Cart-Pole environment, the random policy performs very poorly. Since it selects actions uniformly at random, it has no notion of balance or control, and the pole usually falls after only a few time steps. As a result, episodes terminate quickly and produce very low total rewards:

```
show_policy(cart_pole, random_pi, n_episodes=5, max_steps=500);
```



To **quantitatively assess** a policy on the environment, we define an **evaluation function** that runs the agent for a fixed number of episodes, records the total reward obtained in each one, and returns the average episode reward. This gives us a simple and consistent measure of policy quality, allowing us to compare different strategies under the same conditions:

```
def evaluate(env, policy, episodes=1):

    # collect total rewards per episode
    rewards = []

    for episode in range(episodes):

        # reset the environment
        state, _ = env.reset()
        done = False
        total_reward = 0.0
```

```
        # run one episode
        while not done:
            action = policy(state)
            state, reward, terminated, truncated, _ = env.step(action)
            total_reward += reward
            done = terminated or truncated

        rewards.append(total_reward)

    # return the average reward over episodes
    return float(np.mean(rewards))
```

To establish the numerical baseline, we evaluate the random policy:

```
random_score = evaluate(cart_pole, random_pi, episodes=1000);
print("Random score", random_score);
```

```
Random score 21.921
```

As expected, the random policy achieves a very low average reward on the Cart-Pole environment, usually failing after only a few time steps. This confirms that keeping the pole balanced requires structured decision-making, and gives us a clear benchmark against which to measure learned policies. We can now try to solve the environment using reinforcement learning. However, **tabular methods are not practical** here. The state space is continuous, so a table-based solution would require discretizing the four state variables into a grid of cells. Most of these cells would rarely, if ever, be visited, making the representation inefficient and unreliable. To solve this problem effectively, we need **function approximation**, so that the agent can generalize from the states it has experienced to similar states it has not seen before.

1.2 Neural Fitted Q (NFQ)

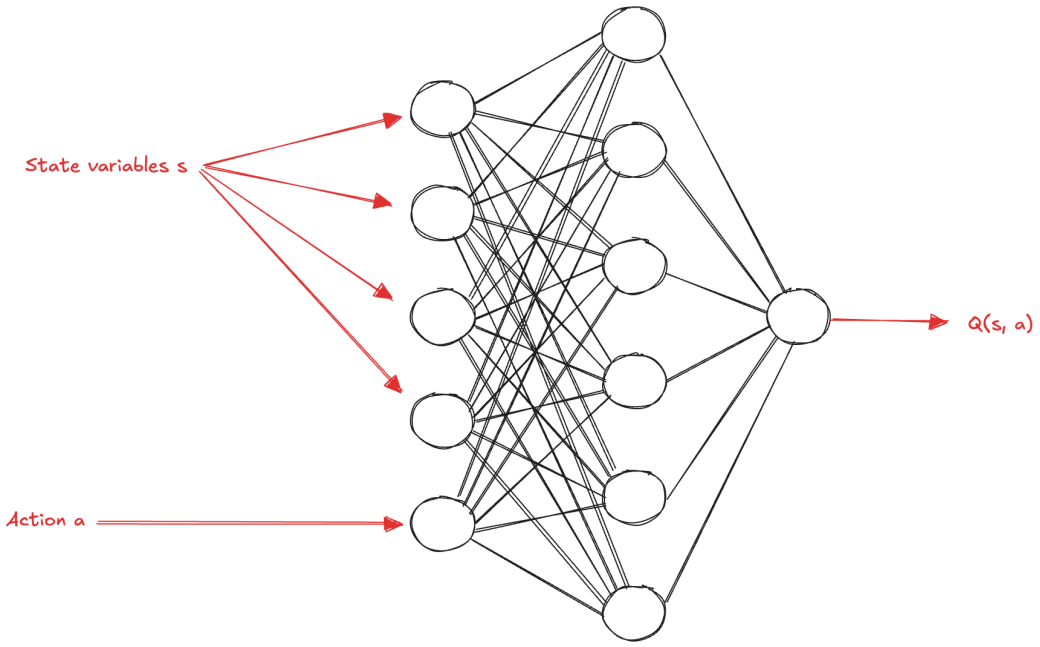
Neural Fitted Q was one of the first reinforcement learning algorithms to successfully use **neural networks as function approximators**. As in the linear case, we denote the approximate action-value function by:

$Q_\theta(s, a)$

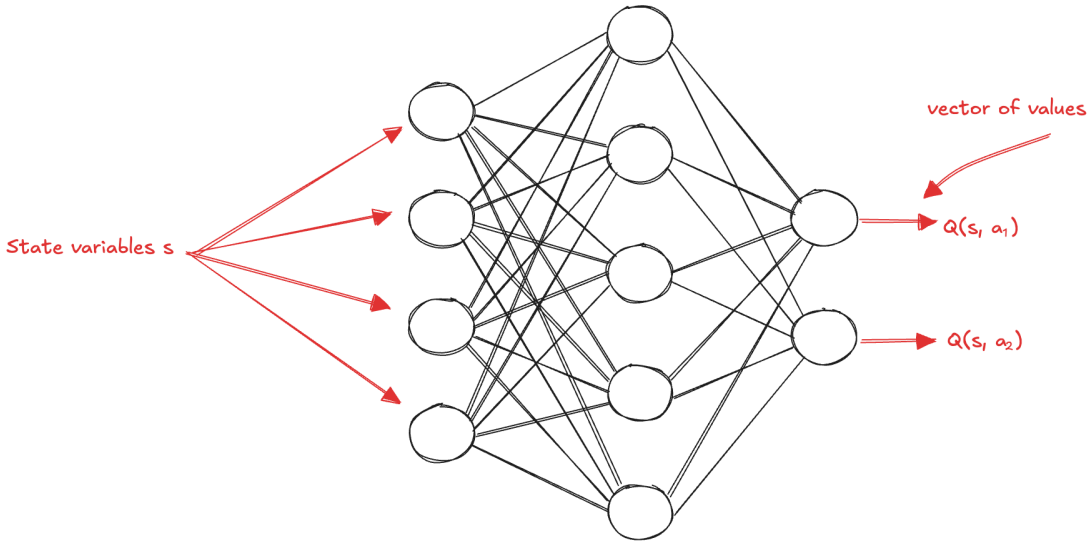
Here, θ denotes the weights of the neural network.

1.2.1 Network Architecture

To design the function approximator, we can start from the tabular representation. In a table, the action-value function is indexed by state-action pairs. A direct neural-network analogue is therefore to provide both the state and the action as input, and to output a single scalar value: the estimated value of that state-action pair.



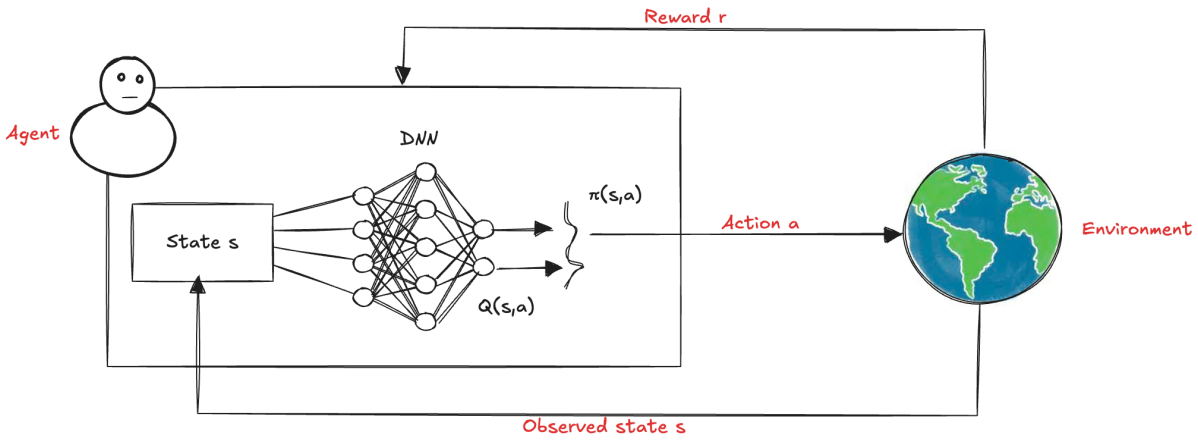
This architecture is valid, but there is a more efficient alternative. Instead of feeding both the state and a candidate action into the network, we feed only the state. The network then outputs one Q-value for each possible action in that state:



This architecture is especially useful with exploration strategies such as epsilon-greedy or softmax, because **a single forward pass gives the values of all available actions**. The state is passed through the network, and the network outputs an estimate of the action-value function for that state.

1.2.2 Universal Approximation

The policy remains implicit, as in Q-learning: the agent acts greedily, or approximately greedily during exploration, with respect to the estimated Q-values. This removes the need to store a separate value for every state-action pair. Instead, the network learns patterns from experience and uses them to generalize across states. This is essential in large or continuous environments, where enumerating all possible states is impossible. The parameters are learned from experience through gradient-based optimization. As the network weights are updated, the Q-value estimates improve, and so does the implicit policy derived from them:



This use use of neural networks as function approximators is supported by the **universal approximation theorem**). The theorem states that a feed-forward network with a single hidden layer and sufficiently many units can approximate any continuous function on a compact set to arbitrary accuracy. However, this is only an existence result. It tells us that a good approximation can exist, but it does not tell us whether gradient descent, applied to our reinforcement learning targets, will actually find it.

We implement the function approximator in PyTorch. For Cart-Pole, a small network is sufficient: we use two hidden layers, both followed by ReLU activations. The input layer has one unit for each state variable, the output layer is linear and has one unit for each action, since the network must return the estimated Q-values directly:

```
import torch

# Define hyperparameters for the neural network
first_hidden_layer = 512;
second_hidden_layer = 128;

def create_network():

      # Define a deep neural network using Sequential:
      # Each layer feeds directly into the next one.
      dnn = torch.nn.Sequential(
            # First fully connected layer maps state inputs to 512 hidden units
            torch.nn.Linear(state_size[0], first_hidden_layer),

            # ReLU activation introduces nonlinearity
            torch.nn.ReLU(),

            # Second fully connected layer (hidden layer with 128 units)
```

```
            torch.nn.Linear(first_hidden_layer, second_hidden_layer),

            # Another ReLU activation
            torch.nn.ReLU(),

            # Output layer: one unit per possible action
            # Produces Q-values
            torch.nn.Linear(second_hidden_layer, action_size)
      )

      # Return the constructed model
      return dnn;

q = create_network();
```

Using the neural network that estimates Q-values, we can define a greedy policy that selects the action with the highest predicted value in the current state. This means acting optimally with respect to the information currently encoded in the network:

```
def nfq_pi(state):

    # convert state to a torch tensor
    state = torch.as_tensor(state, dtype=torch.float32)

    # compute Q-values from the network (no gradient needed here)
    with torch.no_grad():
        q_values = q(state).numpy().squeeze()

    # select greedy action
    action = int(np.argmax(q_values))

    return action
```

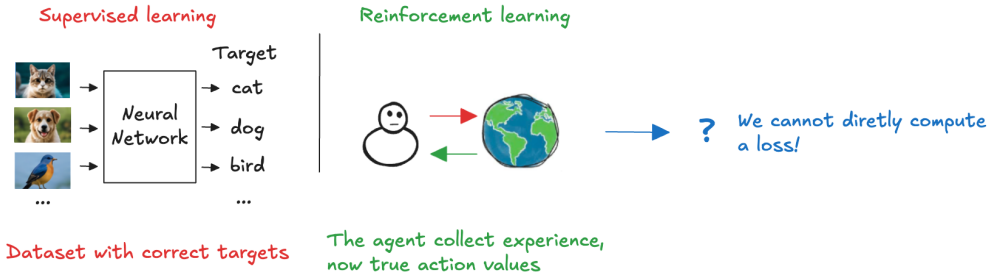
1.2.3 Loss Function

To train a neural network, we need a **loss function**: a measure of how far the network's predictions are from the values we want it to predict. In supervised learning, this is straightforward. We are given a dataset of examples, and each example comes with the correct output. The loss then measures the distance between the network's prediction and the correct target. For regression problems, a common choice is the **mean squared error** (MSE), which averages the

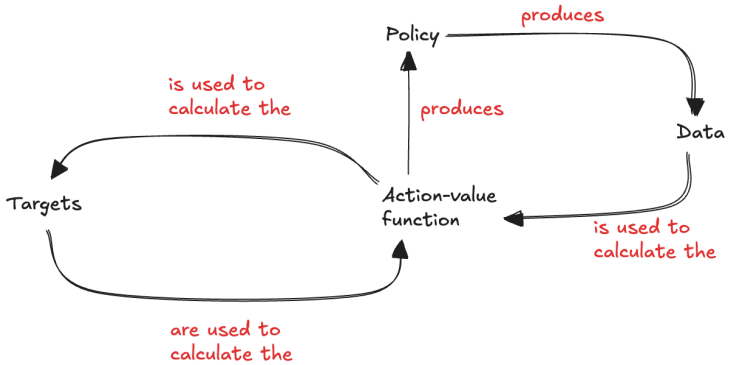
squared differences between predicted and target values. In reinforcement learning, the situation is more difficult for two reasons. First, **we do not start with a fixed dataset**: the agent must collect experience by interacting with the environment. Second, **we do not know the true action values**. Ideally, we would like to minimize the squared distance between our approximation and the optimal action-value function:

$$\mathcal{L}(\theta) = \mathbb{E}_{s,a}[(q_*(s,a) - Q_\theta(s,a))^2]$$

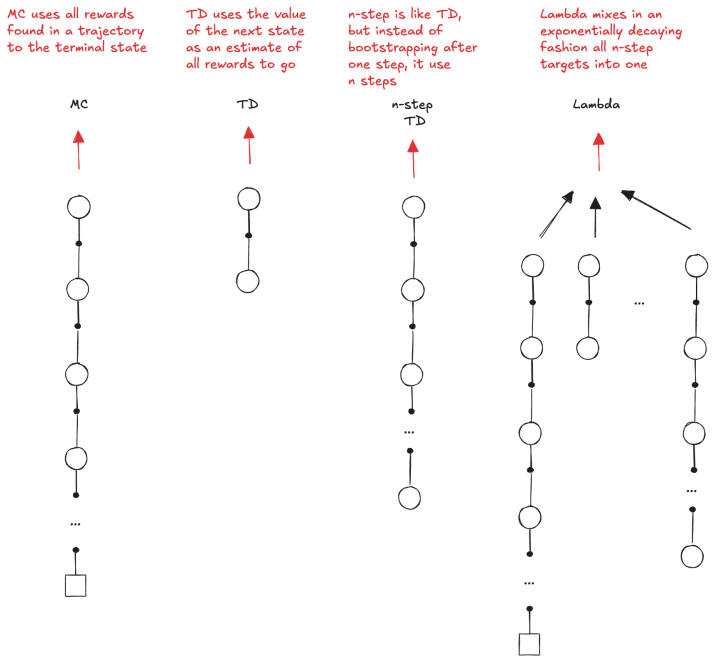
Of course, we do not have access to the optimal function. If we did, there would be nothing left to learn:



Instead, we build training targets using the same **bootstrapping** idea used in Q-learning. After observing a transition, we treat the immediate reward plus the estimated value of the next state as a **temporary target** for the current prediction. Since we are interested in the optimal action-value function, the next-state value is computed by taking the maximum over the available actions. In this sense, the update combines the two parts of generalized policy iteration. The network is **evaluated** by being trained to match the bootstrapped target, and the policy is **improved** because the target assumes greedy action selection in the next state. Thus, policy improvement is not a separate step here: it is built directly into the target through the maximum over actions. However, because we are using **non-linear function approximation**, the convergence guarantees of tabular Q-learning no longer apply. The targets used for training are not fixed labels. They depend on predictions made by the same network, especially through the estimated value of the next state. This creates a circular dependency:



This circular dependency is **problematic**. It makes the learning problem less well behaved than standard supervised learning, because the targets are not fixed: they depend on the network's own current predictions and change as the network changes Nevertheless, we can still use this idea and examine how far it takes us. As we have seen before, there are several possible targets for estimating an action-value function, including Monte Carlo (MC) targets, Temporal-Difference (TD) targets, n-step targets, and lambda targets:



The target can be **on-policy** or **off-policy**, depending on how we bootstrap from the next state. In an on-policy method such as SARSA, the target uses the value of the action that the agent actually takes in the next state. In an off-policy method such as Q-learning, the target uses the value of the best action available in the next state. The corresponding tabular update rules are:

$$Q_{\text{SARSA}}(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

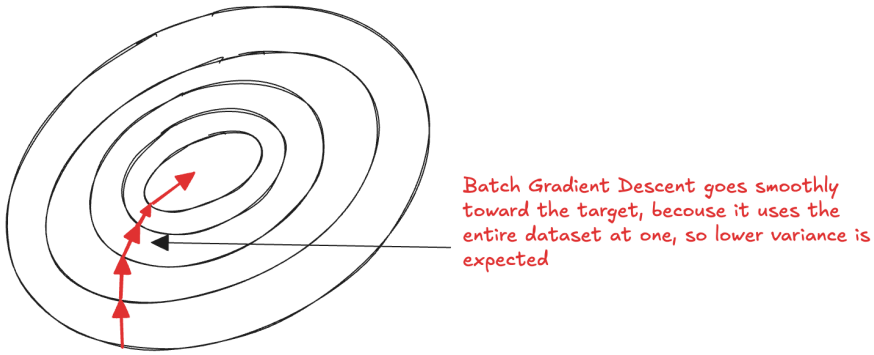
$$Q_{\text{Q-learning}}(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)]$$

NFQ uses the Q-learning target, so it is an **off-policy** method and its loss is:

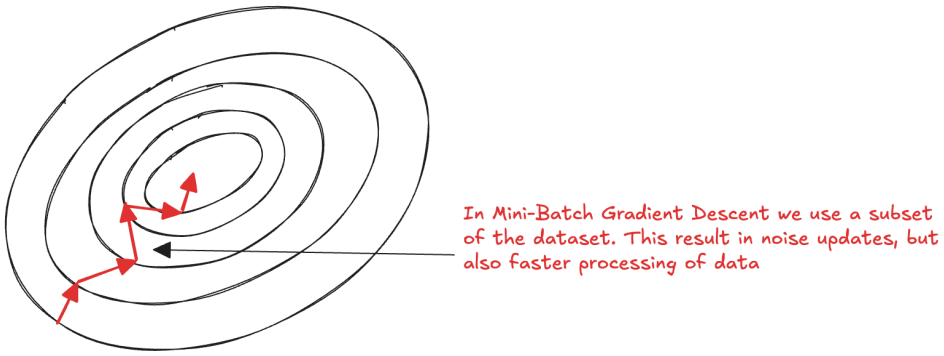
$$\mathcal{L}(\theta) = \mathbb{E}_{s,a,r,s'}\left[\left(r + \gamma \max_{a'} Q_\theta(s', a') - Q_\theta(s, a)\right)^2\right]$$

1.2.4 Optimization

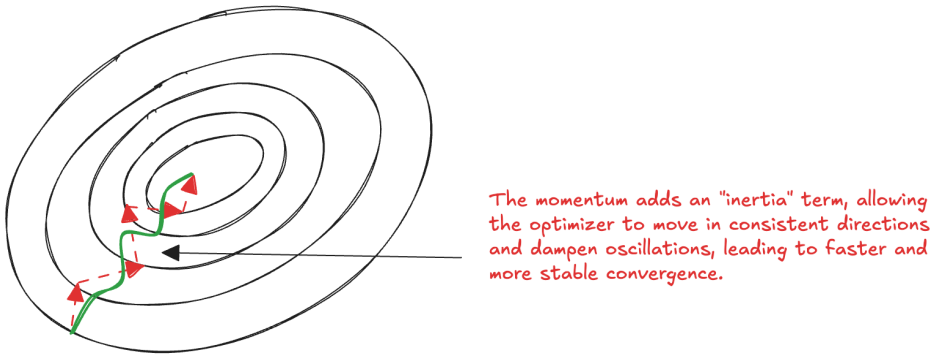
To minimize the loss, we use the same **gradient-based optimizers** commonly used in supervised learning. **Batch gradient descent** computes the gradient using the entire dataset before each parameter update:



Mini-batch gradient descent computes the gradient using only a subset of the data at each update. If the subset contains a single sample, the method becomes stochastic gradient descent. Larger batches usually produce lower-variance updates, but each update is more expensive to compute:



Several optimizers improve on this basic idea. **Momentum** updates the parameters using a moving average of past gradients rather than only the current gradient. **RMSprop** adapts the step size of each parameter by scaling it according to the recent magnitude of its gradients. **Adam** combines both ideas: momentum-like averaging and adaptive per-parameter step sizes.



For our purposes, the key issue is not the optimizer itself, but the assumptions under which these methods are usually analyzed. Standard optimization in supervised learning typically as-

sumes that the samples are **independent and identically distributed (IID)**, and that the targets are **stationary**. In reinforcement learning, neither assumption holds. Consecutive experiences are **strongly correlated**, the **data distribution changes** as the policy changes, and **the targets move** as the network is updated. Moreover, we do not start with a complete dataset, so pure batch gradient descent is not directly applicable. For this reason, the choice of optimizer can matter a great deal. In practice, robust adaptive optimizers such as RMSprop and Adam are often sensible choices for reinforcement learning:

```
learning_rate = 0.007;
optimizer = torch.optim.RMSprop(q.parameters(), lr=learning_rate);
```

All these optimizers require **the gradient of the loss**, and this introduces an important subtlety. In supervised learning, the target values are fixed labels provided in advance. The gradient is therefore computed only with respect to the model's prediction. In NFQ, however, the target is also built using the same network we are training. In principle, this means that the target depends on the parameters too. The loss is:

$$\mathcal{L}(\theta) = \mathbb{E}_{s,a,r,s'}\Big[\big(r + \gamma \max_{a'} Q_\theta(s',a') - Q_\theta(s,a)\big)^2\Big]$$

However, when computing the update, **we treat the bootstrapped target as fixed**:

$$\underbrace{r + \gamma \max_{a'} Q_\theta(s',a')}_{\text{treated as fixed target}}$$

and differentiate only through the current prediction:

$$\underbrace{Q_\theta(s,a)}_{\text{differentiated}}$$

With this convention, the gradient used by NFQ is:

$$\nabla_\theta \mathcal{L}(\theta) = -2\,\mathbb{E}_{s,a,r,s'}\Big[\big(r + \gamma \max_{a'} Q_\theta(s',a') - Q_\theta(s,a)\big)\,\nabla_\theta Q_\theta(s,a)\Big]$$

A gradient descent step then gives the familiar parametric update:

$$\theta \leftarrow \theta + \alpha\,\big(r + \gamma \max_{a'} Q_\theta(s',a') - Q_\theta(s,a)\big)\,\nabla_\theta Q_\theta(s,a)$$

Because we ignore the dependence of the target on the parameters, this is called a **semi-gradient method**. In PyTorch, this is the role of ".detach()" or "torch.no_grad()": they prevent gradients from flowing through the target.

1.2.5 Dataset and training

At this point, we have a neural network, a bootstrapped target, and a loss function. A natural first idea would be to imitate tabular Q-learning as closely as possible: let the agent interact with

the environment, and after each transition, immediately update the network using that single experience. With a table, this works reasonably well because each update affects only one entry. With a neural network, the situation is different. The same parameters are shared across many states and actions, so changing the weights to improve the prediction for one state-action pair can also change the predictions elsewhere in the state-action space. In principle, this shared structure is exactly what we want: it is what allows the network to generalize. In practice, however, online updates can make learning **unstable** and **slow**, because each new experience may partially undo or distort what the network learned from previous ones. The basic idea of NFQ is therefore to move from online updates to an **offline training procedure**. Instead of updating the network after every single transition, we first collect a set of transition experiences and then train the network on the whole dataset. For this reason, we store the agent's experiences in a training set:

```
# define the structured dtype for an experience tuple
experience_type = np.dtype([
    ('state',      np.float32, state_size),   # current state
    ('action',     np.int8),                  # action taken
    ('reward',     np.float32),               # reward received
    ('next_state', np.float32, state_size),   # next state
    ('failure',    np.int8)                   # 1 if next_state is terminal (episode
 ↪  terminated, NOT truncated)
])
```

To train the agent, we must collect batches of experiences by interacting with the environment. During this phase, we use an **exploration strategy** to balance the trade-off between exploring new actions and exploiting what the agent has already learned. The common choice is the ε-greedy policy. We use a constant and rather large $\varepsilon = 0.5$: in the Cart-pole a mostly random behaviour produces short episodes that end in failure, but this is not a problem here, since the transitions that lead to termination are exactly those from which the network must learn that a state is bad, and what we evaluate is the greedy policy, not the exploratory one. A decaying ε, as seen in Chapter 04, is the usual refinement:

```
def epsilon_greedy(state, epsilon=0.5):

    # exploration
    if random.random() < epsilon:
        action = random_pi(state)

    # exploitation
    else:
```

```
        action = nfq_pi(state)

    return action
```

We collect the experience by interacting with the environment. During this data-collection phase, the agent follows an **exploration strategy**, so that it can try different actions instead of always exploiting its current estimates. A common choice is the **epsilon-greedy policy**. This means that the exploratory policy is quite noisy, and many episodes will be short and end in failure. For NFQ, this is acceptable. The transitions that lead to failure are informative: they teach the network **which states and actions are dangerous**. Moreover, the policy we ultimately evaluate is not the exploratory policy, but the greedy policy derived from the learned Q-values. Once a batch of transitions has been collected, we can **train the network** by computing one bootstrapped target for each transition and then fitting the predicted Q-values to those targets for several optimization epochs:

```
def optimize(batch, gamma=0.99, epochs=40):

    # convert experience fields to tensors
    states      = torch.from_numpy(batch["state"].copy()).float()
    actions     = torch.from_numpy(batch["action"].copy()).long()
    rewards     = torch.from_numpy(batch["reward"].copy()).float()
    next_states = torch.from_numpy(batch["next_state"].copy()).float()
    failures    = torch.from_numpy(batch["failure"].copy()).float()

    # Compute the targets once with the current
    # network, then fit the network to them for several epochs
    with torch.no_grad():
        q_next = q(next_states)
        max_q_next = q_next.max(dim=1).values
        target = rewards + gamma * max_q_next * (1.0 - failures)

    # perform multiple epochs of optimization
    for epoch in range(epochs):

        # compute current Q(s, a)
        q_current_all = q(states)
        q_current = q_current_all.gather(1, actions.unsqueeze(1)).squeeze(1)

        # compute the TD error and the mean squared TD error loss
        td_error = target - q_current
        loss = (td_error ** 2).mean()
```

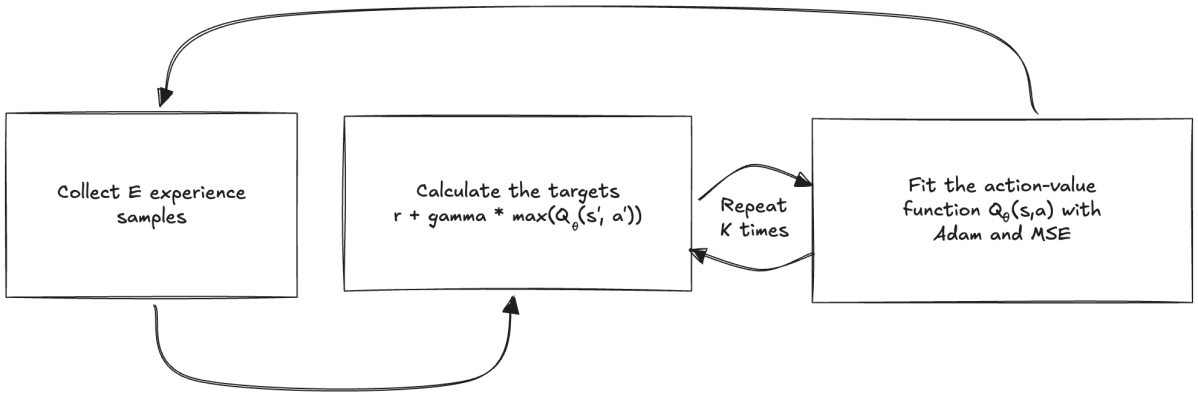
```
        # optimize network parameters
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    # return the final loss as a plain number
    return loss.item()
```

1.2.6 Algorithm

We can now put the pieces together and describe the full training loop:

1. Use a fully connected neural network that takes the state as input and outputs one Q-value for each action
2. Use an epsilon-greedy policy to interact with the environment and collect a batch of experiences
3. Using the current network, compute the off-policy target for each transition in the batch
4. Fit the network to these targets by minimizing the mean squared TD error for several epochs
5. Repeat the process: collect new experiences, compute new targets, and update the network again.




```
def nfq(env, max_episodes, batch_size=1024, epsilon=0.5):

    # create an empty experience batch
    batch = np.empty(batch_size, dtype=experience_type)
    index = 0
```

```
    # track the evaluation scores
    scores = []

    for episode in range(max_episodes):

        # reset the environment
        state, _ = env.reset()
        done = False

        while not done:

            # interact with the environment using the exploratory policy
            action = epsilon_greedy(state, epsilon)
            next_state, reward, terminated, truncated, _ = env.step(action)
            done = terminated or truncated

            # store the transition
            batch[index] = (state, action, reward, next_state, terminated)
            index += 1

            # once the batch is full, train the network and clear the batch
            if index == batch_size:
                optimize(batch)
                batch = np.empty(batch_size, dtype=experience_type)
                index = 0

            state = next_state

        # evaluate the greedy policy after each episode
        score = evaluate(env, nfq_pi, episodes=10)
        scores.append(score)

        print(f"Episode {episode + 1:03}, score {score:05.1f}", end="\r", flush=True)

    return scores
```

1.2.7 Solve Cart-Pole

We can apply the algorithm to the Cart-pole environment:

```
scores = nfq(cart_pole, max_episodes=280)
```

```
Episode 280, score 500.0
```

We can evaluate the performance of the learned agent versus the random one:

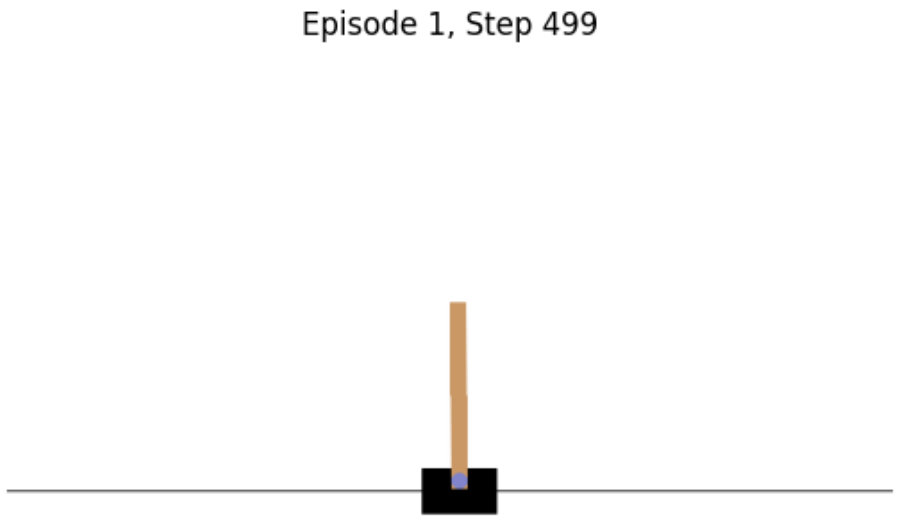
```
agent_score = evaluate(cart_pole, nfq_pi, episodes=10);

print("Random score", random_score);
print("Agent score", agent_score);
```

```
Random score 21.921
Agent score 500.0
```

And we can see the agent in action:

```
show_policy(cart_pole, nfq_pi, n_episodes=1, max_steps=500);
```



The trained agent can balance the pole for a considerable amount of time, but this result was obtained by choosing a number of training steps that gives good performance; with different

choices, the outcome can change substantially. A single run is therefore not enough to understand **how the algorithm learns**, or **how reliable the result is**. To study the learning process, we need to track performance over time in a controlled experiment. We can do this by running several independent training sessions and recording the average score reached after a fixed number of episodes. By repeating the experiment with different random seeds, we can assess how stable the algorithm is and how sensitive it is to random initialization and exploration. This allows us to distinguish genuine learning behavior from a lucky run, and to evaluate whether the algorithm converges reliably across repetitions.

```
import random
from pathlib import Path

import joblib
import numpy as np
import torch


def experiment(
    env,
    train,
    setup,
    max_episodes,
    name,
    seeds=(12, 34, 56, 78, 90),
    smoothing_window=25,
):

    results_path = Path("temp") / f"{name}_results.joblib"

    # load saved results if they already exist
    if results_path.exists():
        experiment_results = joblib.load(results_path)

        n_points = len(experiment_results["mean_score"])
        final_mean = experiment_results["mean_score"][-1]
        final_min = experiment_results["min_score"][-1]
        final_max = experiment_results["max_score"][-1]

        print(f"Loaded saved results from {results_path}")
        print(f"Smoothed points: {n_points}")
        print(f"Smoothing window: {experiment_results['smoothing_window']}")
        print(f"Final score: mean={final_mean:.1f}, range=[{final_min:.1f},
        ↪  {final_max:.1f}]")
```

```
        return experiment_results

    results = []

    for seed in seeds:
        print(f"Experiment seed: {seed}")

        # set seeds for reproducibility
        random.seed(seed)
        np.random.seed(seed)
        torch.manual_seed(seed)

        env.reset(seed=seed)
        env.action_space.seed(seed)

        # initialize networks, optimizer, replay memory, etc.
        setup()

        # train the agent
        scores = train(env, max_episodes=max_episodes)

        # smooth the learning curve
        kernel = np.ones(smoothing_window) / smoothing_window
        scores = np.convolve(scores, kernel, mode="valid")

        results.append(scores)
        print()

    results = np.array(results)

    experiment_results = {
        "max_score": np.max(results, axis=0),
        "min_score": np.min(results, axis=0),
        "mean_score": np.mean(results, axis=0),
        "smoothing_window": smoothing_window,
        "seeds": seeds,
        "max_episodes": max_episodes,
    }

    results_path.parent.mkdir(exist_ok=True)
    joblib.dump(experiment_results, results_path)
```

```
    return experiment_results
```

We now define the hyperparameters that control both the training procedure and the neural network architecture. These values are different from those used in the initial run: the first hidden layer is smaller, the learning rate is much lower, and the number of optimization epochs per batch is reduced. These choices were selected using a **grid search**:

```
# define the experiment setup and hyperparameters
gamma = 0.99                # discount factor
learning_rate = 0.0005      # optimizer step size
batch_size = 1024           # number of experiences per batch
epochs = 16                 # optimization epochs per batch
epsilon = 0.5               # exploration rate
first_hidden_layer = 256    # size of the first hidden layer
second_hidden_layer = 128   # size of the second hidden layer

def setup_nfq():
    global q, optimizer
    q = create_network()
    optimizer = torch.optim.RMSprop(q.parameters(), lr=learning_rate)

# run the experiment
nfq_results = experiment(
    env=cart_pole,
    train=nfq,
    setup=setup_nfq,
    max_episodes=1500,
    name="nfq",
)
```

```
Loaded saved results from temp/nfq_results.joblib
Smoothed points: 1476
Smoothing window: 25
Final score: mean=335.1, range=[150.2, 479.6]
```

After running the experiment, we plot the learning curve by showing the mean score across seeds, together with the range between the best and worst run. This gives a quick visual summary of both performance and variability:

```python
def plot_scores(results, title="NFQ score"):

    # the smoothed curves start after the first smoothing window
    offset = results.get("smoothing_window", 1)
    episodes = np.arange(len(results["mean_score"])) + offset

    plt.figure(figsize=(12, 6))
    plt.title(title)
    plt.xlabel("Episodes")
    plt.ylabel("Score")

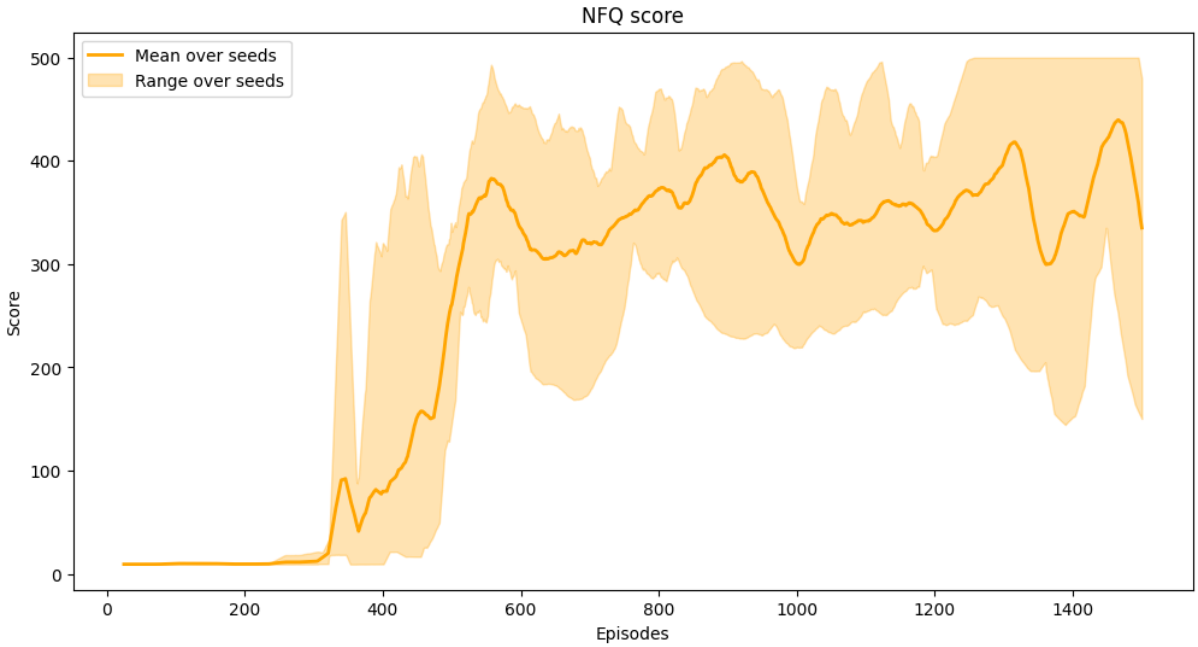
    plt.plot(
        episodes,
        nfq_results["mean_score"],
        color="orange",
        linewidth=2,
        label="Mean over seeds",
    )

    plt.fill_between(
        episodes,
        nfq_results["min_score"],
        nfq_results["max_score"],
        color="orange",
        alpha=0.3,
        label="Range over seeds",
    )

    plt.legend(loc="upper left")
    plt.show()


# reload saved results without retraining
# nfq_results = joblib.load("./temp/nfq_results.joblib")

plot_scores(nfq_results)
```



The NFQ experiment shows that the algorithm can learn a useful policy for Cart-Pole, but also that **the learning process is unstable**. At the beginning, the average score stays close to the random-policy baseline. After a few hundred episodes, performance improves sharply, showing that the network has started to discover a policy that can keep the pole balanced for longer. However, **the improvement is not smooth**. The mean curve fluctuates substantially, and the shaded region between the best and worst seeds is wide. This means that different runs, initialized with different random seeds, learn at different speeds and sometimes reach very different levels of performance. In some runs the agent solves the task almost perfectly, while in others performance remains much lower or deteriorates after improving. Overall, NFQ is able to learn in this simple environment, but the plot also shows that **it is not reliable**. A good result is possible, but it is not guaranteed across seeds. This variability suggests that the algorithm is sensitive to initialization, exploration, and the particular transitions collected during training.

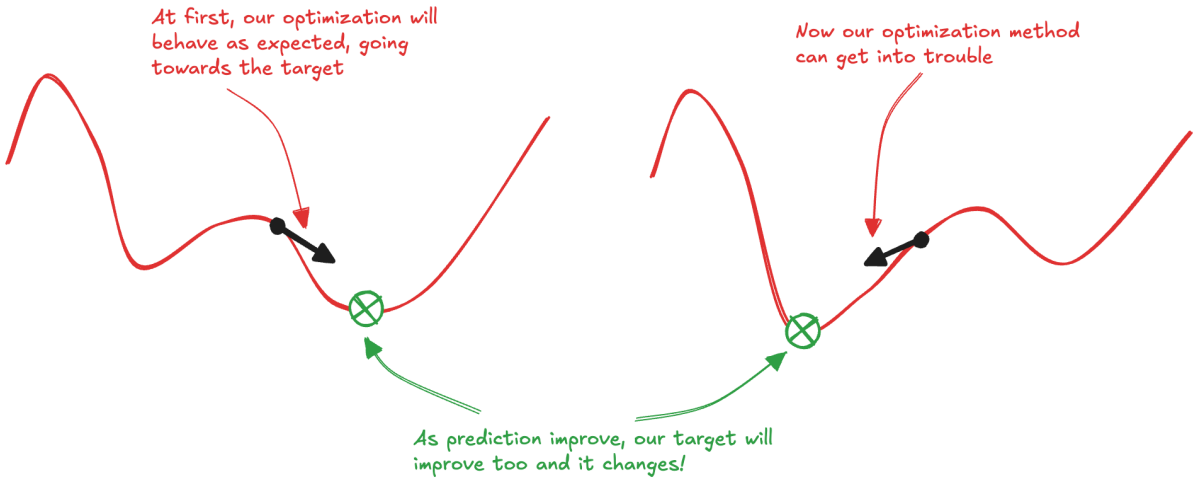
1.3 Deep Q-Networks (DQN)

NFQ shows that neural networks can approximate action-value functions, but it also reveals why combining value-based reinforcement learning with function approximation is delicate. The agent is not learning from a fixed dataset with fixed labels. It collects its own data, updates its policy while learning, and builds its targets from the same value estimates it is trying to improve. **Deep Q-Networks**, or DQN, were introduced to make this idea **more stable and scalable**. They keep the core structure of Q-learning with a neural network, but add two crucial mechanisms: a replay buffer, which reduces the correlation between consecutive samples, and

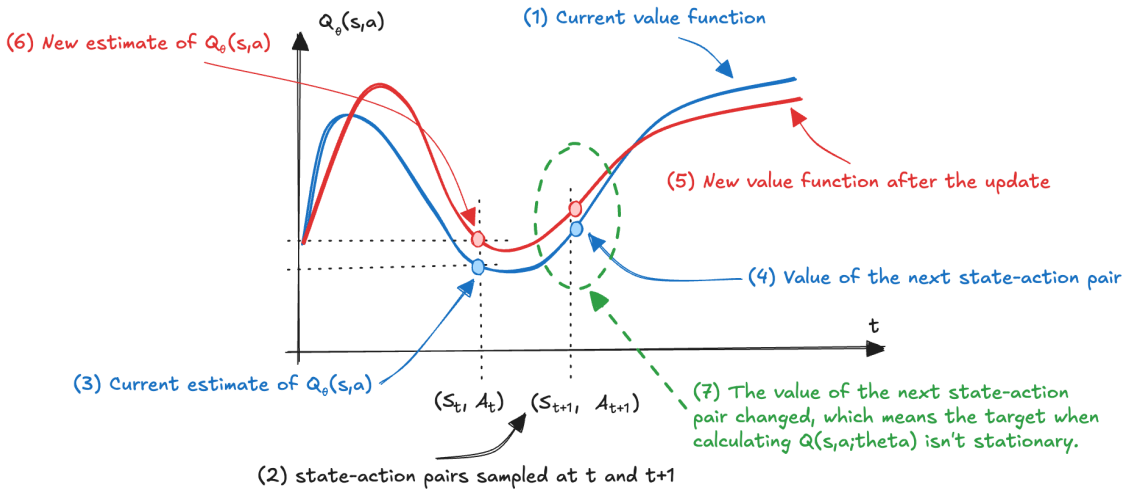
a target network, which makes the bootstrapped targets more stable.

1.3.1 Deadly Triad

NFQ exposes two fundamental difficulties that appear whenever value-based reinforcement learning is combined with function approximation. The first difficulty is **non-stationary targets**. In supervised learning, targets are fixed labels. In value-based reinforcement learning, however, targets are built from the network's own predictions. When the network is updated, the estimated value of the next state changes as well, and therefore the target changes too. The network is trying to chase a target that moves while it is being learned. NFQ reduces this instability by using fitted batch learning: targets are computed once for a batch and then kept fixed during the optimization phase. This makes each fitting step more similar to supervised learning. However, the targets still change from one batch to the next, because they are recomputed using the updated network.



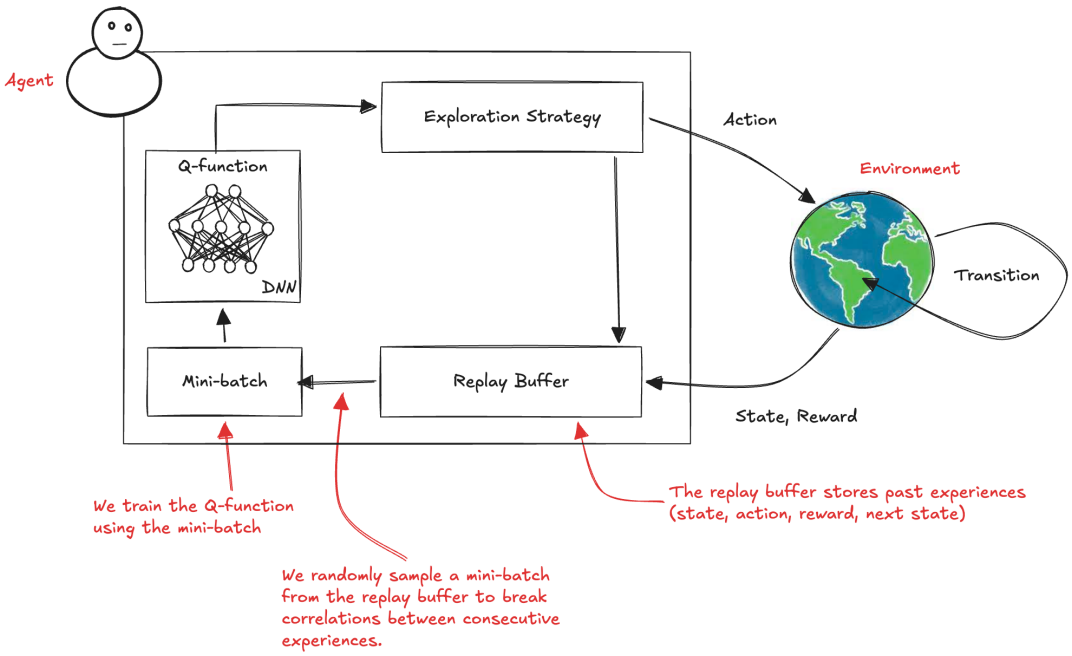
The second difficulty is **correlated samples**. Gradient-based optimizers are usually designed for data that is approximately independent and identically distributed. In reinforcement learning, experience is collected online, along trajectories. The transition observed at time step t+1 is strongly related to the transition observed at time step t. Moreover, the data distribution changes over time because the policy used to collect experience changes as the agent learns.



These two problems are symptoms of a deeper source of instability known as the **deadly triad**. Instability can arise when three ingredients are combined: **function approximation**, **bootstrapping**, and **off-policy learning**. NFQ contains all three. It uses a neural network as a function approximator, it bootstraps because its targets depend on current value estimates, and it is off-policy because the data is collected with an exploratory policy while the targets are computed using greedy action selection.

1.3.2 Experience Replay

In NFQ, the network is trained on a batch of recently collected transitions. Even if the batch contains many samples, those samples are still **strongly correlated**, because they usually come from the same few trajectories. As a result, the optimizer repeatedly sees data that are too similar and too dependent on the agent's most recent behavior. **Experience replay** addresses this problem by storing past transitions in a **replay memory**. Instead of training only on the most recent batch, the agent keeps **a large collection of experiences** gathered over time. At each update, it **samples a mini-batch uniformly at random from this memory**.



This has two important effects. First, the **mini-batches become more diverse**, because they contain transitions from different episodes and different moments of training. Second, **the samples look more like the independent and identically distributed data** assumed by standard optimizers. They are **not truly IID**, but random sampling from a large memory reduces the strongest temporal correlations and makes training more stable. Compared with NFQ, the main change is therefore not the target itself, but where the training samples come from. Instead of using only the most recent online experience, DQN trains on transitions sampled uniformly from the replay buffer:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim U(\mathcal{D})}\Big[\big(r + \gamma \max_{a'} Q_\theta(s',a') - Q_\theta(s,a)\big)^2\Big]$$

In practice, replay memory must be implemented carefully. With high-dimensional observations, such as images, **a naive buffer can quickly run into memory limits**. For example, if each state is represented by a stack of recent frames, storing every full state separately would duplicate many frames and waste memory. For Cart-Pole, the observations are small, so a simple implementation is enough. We use a **circular buffer**: a fixed-size array that stores new transitions until it is full, and then overwrites the oldest transitions with the newest ones. This solves the fixed-size storage problem. In image-based environments, more efficient implementations usually store each frame only once and reconstruct stacked states when sampling.

```
# create the replay memory
memory_size = 100000
```

```
replay_memory = {
    "size": memory_size,
    "buffer": np.empty(memory_size, dtype=experience_type),
    "index": 0,    # position where the next transition will be stored
    "entries": 0,  # number of transitions currently stored
}
```

Each time the agent observes a transition, we insert it into the replay memory. When the buffer is full, the oldest transitions are overwritten in circular order:

```
def store_experience(experience):

    # store the transition at the current position
    replay_memory["buffer"][replay_memory["index"]] = experience

    # update the number of stored transitions
    replay_memory["entries"] = min(
        replay_memory["entries"] + 1,
        replay_memory["size"],
    )

    # move to the next position, wrapping around when the buffer is full
    replay_memory["index"] = (replay_memory["index"] + 1) % replay_memory["size"]
```

Once the replay memory contains enough transitions, we can draw a random mini-batch from it and use that batch to update the network:

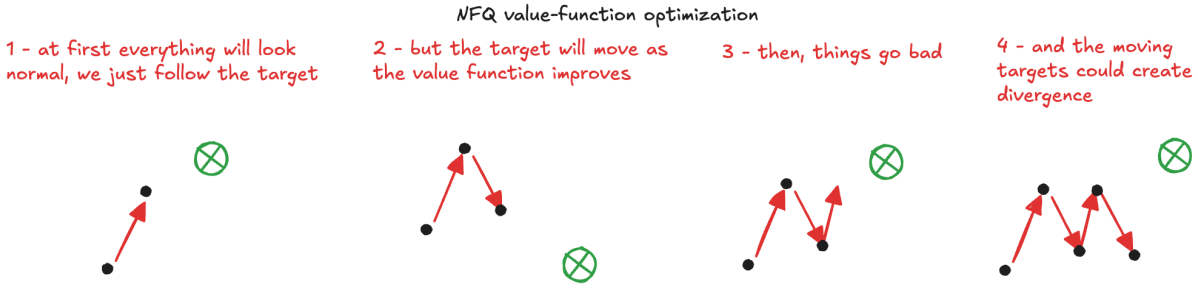
```
def sample_experiences(batch_size=32):

    # sample transitions uniformly at random
    indices = np.random.choice(
        replay_memory["entries"],
        batch_size,
        replace=False,
    )

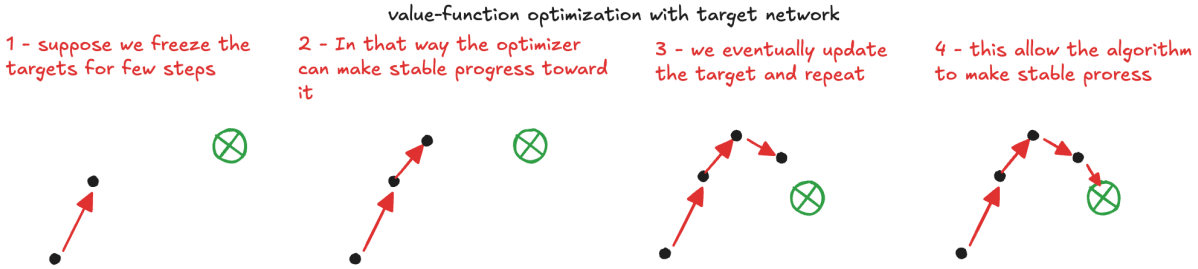
    return replay_memory["buffer"][indices]
```

1.3.3 Target Networks

The replay buffer reduces the correlation between training samples, but it does not solve the second problem of value-based deep reinforcement learning: the targets are still non-stationary:



The issue is that the same network is used both to make the current prediction and to construct the target. When the network is updated, the prediction changes, but the target can change as well, because it depends on the estimated value of the next state. The optimizer is therefore trying to move toward a target that keeps moving. A **target network** makes this process more stable. The idea is to keep two copies of the same neural network. The first is the **online network** and it is updated at every training step. The second is the **target network** and it is used only to compute the bootstrapped targets:



The target network is not a different architecture. It is a delayed copy of the online network. Every few steps, we copy the weights of the online network into the target network. Between two copies, the target network remains fixed, so the targets remain more stable. **This turns training into a sequence of smaller supervised learning problems**: for a while, the labels are fixed, the online network moves toward them, and only later are the labels updated. With a target network, the DQN loss becomes:

$$\mathcal{L}(\theta_{\text{online}}) = \mathbb{E}_{(s,a,r,s')\sim U(\mathcal{D})}\Big[\big(r + \gamma \max_{a'} Q_{\theta_{\text{target}}}(s',a') - Q_{\theta_{\text{online}}}(s,a)\big)^2\Big]$$

The semi-gradient idea is the same as in NFQ. The bootstrapped target, now computed with the frozen target network, is treated as a constant. Freezing the target network does not make the targets perfectly correct, but it makes them stable for long enough that optimization can make meaningful progress. To implement this idea, we use the same network architecture as

before and create two instances of it, one online network for learning, and one target network for computing targets:

```
online_q = create_network();
target_q = create_network();
```

The online network is the one whose parameters are updated during training, so we attach an optimizer to it:

```
learning_rate = 0.007;
optimizer = torch.optim.RMSprop(online_q.parameters(), lr=learning_rate);
```

The target network is not optimized directly. Instead, we periodically **copy the weights** of the online network into it:

```
def update_target():
    # copy the parameters of the online network into the target network (hard update)
    target_q.load_state_dict(online_q.state_dict());
```

After training, the online network defines the learned policy:

```
def dqn_pi(state):
    # convert the state into a tensor
    state = torch.as_tensor(state, dtype=torch.float32);

    # compute Q-values from the network
    q_values = online_q(state).detach().numpy().squeeze()

    # select greedy action
    action = int(np.argmax(q_values))

    # return the action
    return action;
```

Using a target network makes learning more stable because the targets remain fixed for several updates. This gives the online network time to move toward them before the target values are refreshed. The trade-off is that the targets are slightly outdated. If we update the target network too rarely, **learning can become slow or biased** toward old estimates. If we update it too often, **we lose the stabilizing effect**. The **update frequency** therefore controls the balance between

stability and speed. The optimization step now uses the target network to compute the value of the next state, and the online network to compute the value of the current state-action pair:

```
def optimize(gamma=0.99, batch_size=32):

    # sample a mini-batch from the replay memory
    batch = sample_experiences(batch_size)

    # convert experience fields to tensors
    states      = torch.from_numpy(batch["state"].copy()).float()
    actions     = torch.from_numpy(batch["action"].copy()).long()
    rewards     = torch.from_numpy(batch["reward"].copy()).float()
    next_states = torch.from_numpy(batch["next_state"].copy()).float()
    failures    = torch.from_numpy(batch["failure"].copy()).float()

    # compute the target with the target network
    with torch.no_grad():
        q_next = target_q(next_states)
        max_q_next = q_next.max(dim=1).values
        target = rewards + gamma * max_q_next * (1.0 - failures)

    # compute the current estimate with the online network
    q_current_all = online_q(states)
    q_current = q_current_all.gather(1, actions.unsqueeze(1)).squeeze(1)

    # compute the mean squared TD error
    td_error = target - q_current
    loss = (td_error ** 2).mean()

    # update the online network
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    return loss.item()
```

1.3.4 Exploration Strategies

Exploration is essential in reinforcement learning, because the agent must try different actions in order to discover which ones lead to better long-term outcomes. In NFQ, we used an epsilon-greedy strategy with a fixed exploration rate. This is simple, but it keeps the same amount of randomness throughout training. For DQN, we use a **decaying epsilon-greedy strategy**. Training

begins with a high value of exploration, so the agent explores widely when it knows little about the environment. As training progresses, it is gradually reduced, and the agent increasingly exploits the knowledge encoded in its Q-network. This schedule is still simple, but it is usually more effective than keeping exploration fixed. Other exploration strategies are possible and even small changes in exploration hyperparameters can have a large effect on performance.

```
# define the exploration schedule
epsilon_max = 1.0
epsilon_min = 0.01
epsilon_decay_steps = 10_000

epsilons = np.logspace(
    start=np.log10(epsilon_max),
    stop=np.log10(epsilon_min),
    num=epsilon_decay_steps,
)
```

```
def epsilon_greedy(state, step):

    # get the exploration rate for the current step
    if step < epsilon_decay_steps:
        epsilon = epsilons[step]
    else:
        epsilon = epsilon_min

    # explore
    if random.random() < epsilon:
        return random_pi(state)

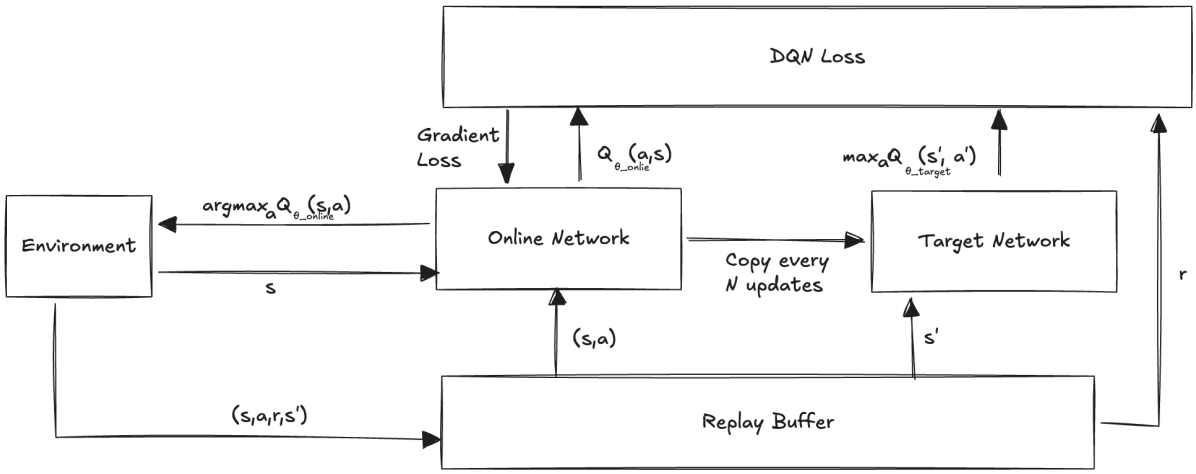
    # exploit
    return dqn_pi(state)
```

1.3.5 Algorithm

DQN is among the most influential algorithms in deep reinforcement learning. It showed that a neural network could learn effective value functions directly from **high-dimensional observations**, such as raw pixels. This result led to **human-level, and in some cases superhuman, performance** on several games in the **Atari benchmark**. We now implement the full training loop. Compared with the simplified NFQ implementation, DQN introduces the three key changes: a

replay buffer, a **target network**, and a **decaying ε-greedy exploration strategy**. The algorithm proceeds as follows:

1. Collect a transition by interacting with the environment and store it in the replay buffer
2. Sample a mini-batch uniformly at random from the replay buffer
3. Use the target network to compute the next-state Q-values, and the online network to compute the current-state Q-values
4. Compute the loss and update only the online network
5. Periodically copy the online network weights into the target network



We can now write the DQN training loop. The agent first fills the replay buffer with a minimum number of transitions, then alternates between collecting new experience, sampling mini-batches for optimization, and periodically synchronizing the target network with the online network:

```
def dqn(env, max_episodes, memory_start_size=1000, target_update_steps=100):

    scores = []
    step = 0

    # initialize the target network as a copy of the online network
    target_q.load_state_dict(online_q.state_dict())

    for episode in range(max_episodes):

        state, _ = env.reset()
        done = False

        while not done:
```

```
            # collect one transition
            action = epsilon_greedy(state, step)
            next_state, reward, terminated, truncated, _ = env.step(action)
            done = terminated or truncated

            experience = (state, action, reward, next_state, terminated)
            store_experience(experience)

            # update the online network once the replay memory is large enough
            if replay_memory["entries"] > memory_start_size:
                optimize()

                # periodically update the target network
                if step % target_update_steps == 0:
                    update_target()

            state = next_state
            step += 1

        # evaluate the greedy policy after each episode
        score = evaluate(env, dqn_pi, episodes=10)
        scores.append(score)

        print(f"Episode {episode + 1:03}, score {score:05.1f}", end="\r", flush=True)

    return scores
```

1.3.6 Solve Cart-Pole

We can now apply DQN to the Cart-Pole environment. As before, the agent's performance depends strongly on the choice of hyperparameters, so careful tuning is important.

```
# train the DQN agent
scores = dqn(cart_pole, max_episodes=200)
```

```
Episode 200, score 218.2
```

From this first run, DQN appears to learn much faster than NFQ on the Cart-Pole environment.However, a single run is not enough to draw a reliable conclusion, because performance

can depend strongly on the random seed and on the particular experiences collected during training. To compare the two algorithms more carefully, we run the same type of experiment used for NFQ: several independent training runs with different random seeds, followed by an evaluation of the average learning curve and its variability.

Then we run the experiment for the DQN agent on the cart-pole environment:

```
# define the experiment setup and hyperparameters
gamma = 0.99                # discount factor
memory_size = 100_000       # replay memory size
memory_start_size = 1_000   # minimum transitions before training
batch_size = 32             # number of transitions per mini-batch
target_update_steps = 100   # steps between target network updates
first_hidden_layer = 512    # size of the first hidden layer
second_hidden_layer = 128   # size of the second hidden layer
learning_rate = 0.007       # optimizer step size
epsilon_max = 1.0           # initial exploration rate
epsilon_min = 0.01          # final exploration rate
epsilon_decay_steps = 10_000 # number of steps used to decay epsilon

def setup_dqn():

    global online_q, target_q, optimizer, replay_memory, epsilons

    # create fresh networks and optimizer
    online_q = create_network()
    target_q = create_network()
    optimizer = torch.optim.RMSprop(online_q.parameters(), lr=learning_rate)

    # create a fresh replay memory
    replay_memory = {
        "size": memory_size,
        "buffer": np.empty(memory_size, dtype=experience_type),
        "index": 0,
        "entries": 0,
    }

    # create the exploration schedule
    epsilons = np.logspace(
        start=np.log10(epsilon_max),
        stop=np.log10(epsilon_min),
        num=epsilon_decay_steps,
    )
```

```
# run the experiment
dqn_results = experiment(
    env=cart_pole,
    train=dqn,
    setup=setup_dqn,
    max_episodes=250,
    name="dqn",
)
```

```
Experiment seed: 12
Episode 250, score 214.5
Experiment seed: 34
Episode 250, score 113.5
Experiment seed: 56
Episode 250, score 141.0
Experiment seed: 78
Episode 250, score 046.5
Experiment seed: 90
Episode 250, score 101.8
```

To compare the two algorithms directly, we plot their average learning curves together with the range of scores observed across random seeds:

```
def plot_comparison(
    first_results,
    second_results,
    first_label="DQN",
    second_label="DDQN",
    first_color="blue",
    second_color="red",
):

    first_episodes = np.arange(len(first_results["mean_score"]))
    second_episodes = np.arange(len(second_results["mean_score"]))

    plt.figure(figsize=(12, 6))
    plt.title(f"{second_label} vs {first_label}")
    plt.xlabel("Episodes")
    plt.ylabel("Score")
```

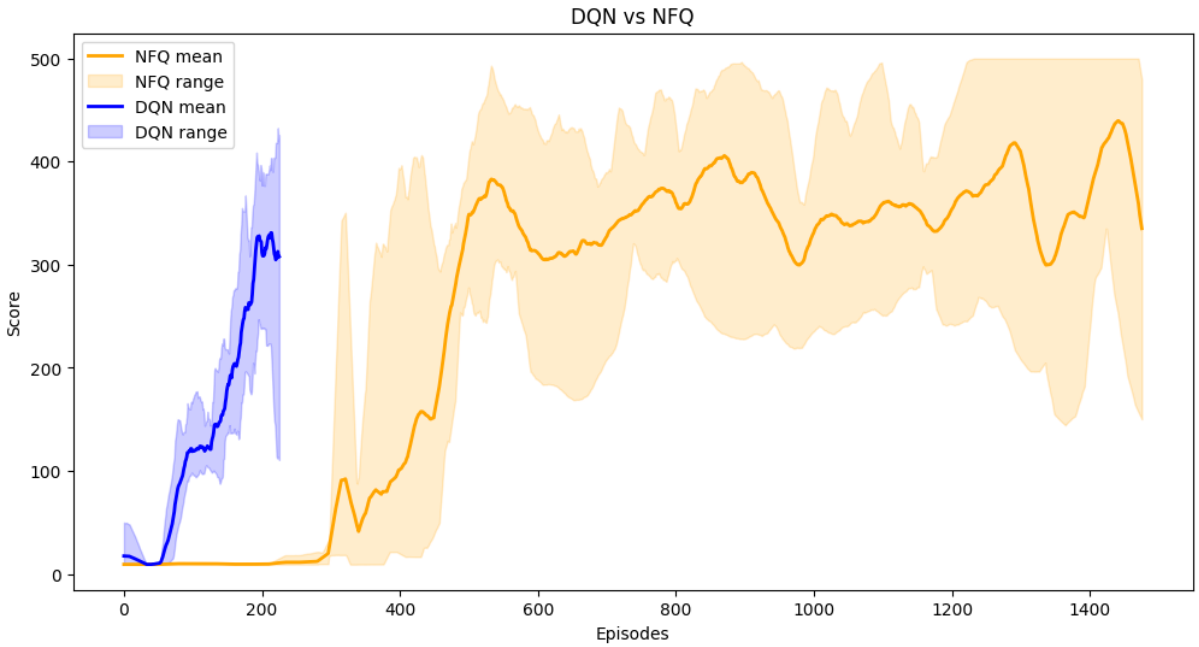
```
    # plot the first algorithm
    plt.plot(
        first_episodes,
        first_results["mean_score"],
        color=first_color,
        linewidth=2,
        label=f"{first_label} mean",
    )
    plt.fill_between(
        first_episodes,
        first_results["min_score"],
        first_results["max_score"],
        color=first_color,
        alpha=0.20,
        label=f"{first_label} range",
    )

    # plot the second algorithm
    plt.plot(
        second_episodes,
        second_results["mean_score"],
        color=second_color,
        linewidth=2,
        label=f"{second_label} mean",
    )
    plt.fill_between(
        second_episodes,
        second_results["min_score"],
        second_results["max_score"],
        color=second_color,
        alpha=0.20,
        label=f"{second_label} range",
    )

    plt.legend()
    plt.show()
```

```
plot_comparison(
    nfq_results,
    dqn_results,
    first_label="NFQ",
```

```
    second_label="DQN",
    first_color="orange",
    second_color="blue",
)
```



The comparison suggests that **DQN learns much faster than NFQ** on Cart-Pole. The DQN curve starts improving within the first few dozen episodes, while NFQ needs several hundred episodes before its average score rises substantially. This is consistent with the role of experience replay and the target network: the replay buffer provides more diverse mini-batches, and the target network makes the bootstrapped targets more stable. At the same time, the shaded region shows that the **variability across seeds is still large**. Some runs improve quickly, while others remain much weaker or fluctuate substantially. This is an important reminder that value-based deep reinforcement learning can remain sensitive to initialization, exploration, and the particular sequence of experiences collected during training.

1.3.7 Double Deep Q-Learning (DDQN)

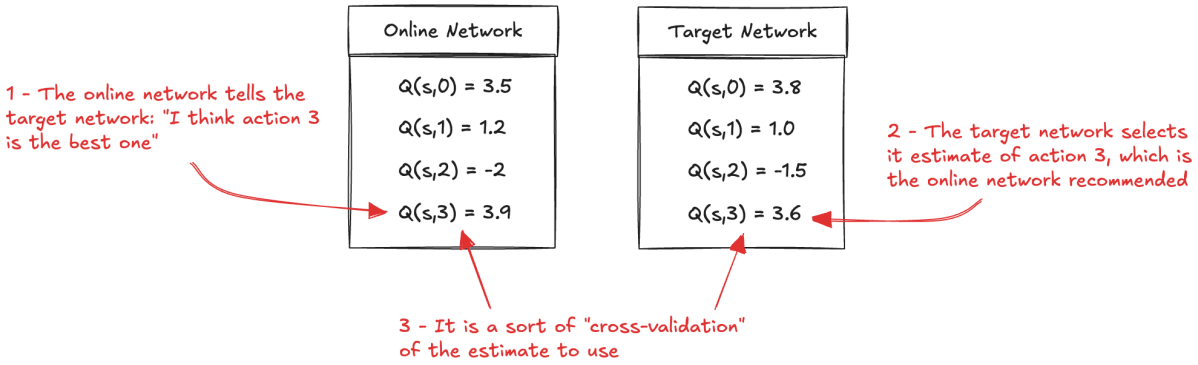
Q-learning tends to **overestimate action values**, a phenomenon known as **maximization bias**. This bias appears because the target uses a maximum over estimated values. When those estimates are noisy, the maximum is more likely to select an action whose value has been overestimated. DQN has the same problem, because it uses the same off-policy target with a max

operator. To see where the bias comes from, we can rewrite the maximum explicitly as an action selection step followed by an action evaluation step:

$$\max_{a'} Q_{\theta_{\text{target}}}(s', a') = Q_{\theta_{\text{target}}}(s', \arg\max_{a'} Q_{\theta_{\text{target}}}(s', a'))$$

This expression shows that the max operation is really doing two things at once. First, it selects the action with the highest estimated value in the next state. Then, it uses the value of that same action to build the target. In standard DQN, both steps are performed using the target network. The same estimates are used both to choose the action and to evaluate it. If those estimates contain noise, the selected action is likely to be one whose value is too high, and this produces a positive bias in the target. **Double Q-learning** addresses this problem by separating action selection from action evaluation. One estimator selects the action, while another estimator evaluates that action. In deep reinforcement learning, we can implement this idea without adding extra networks. DQN already has two networks: the online network and the target network. DDQN uses them as follows:

- the **online network** selects the best next action
- the **target network** evaluates the value of that selected action



The DDQN target therefore differs from the DQN target only in how the next-state action is selected. The online network chooses the action, while the target network assigns its value:

$$\mathcal{L}(\theta_{\text{online}}) = \mathbb{E}_{(s,a,r,s')\sim U(\mathcal{D})}\Big[\big(r + \gamma Q_{\theta_{\text{target}}}(s', \arg\max_{a'} Q_{\theta_{\text{online}}}(s', a')) - Q_{\theta_{\text{online}}}(s, a)\big)^2\Big]$$

This small change reduces overestimation. The online network may still choose an action whose value it overestimates, but the target value is computed by the separate target network. As a result, the action selection and action evaluation errors are partially decoupled. We can now modify the optimization step to implement this idea:

```
def optimize(gamma=0.99, batch_size=32):

    # sample a mini-batch from the replay memory
```

```
    batch = sample_experiences(batch_size)

    # convert experience fields to tensors
    states      = torch.from_numpy(batch["state"].copy()).float()
    actions     = torch.from_numpy(batch["action"].copy()).long()
    rewards     = torch.from_numpy(batch["reward"].copy()).float()
    next_states = torch.from_numpy(batch["next_state"].copy()).float()
    failures    = torch.from_numpy(batch["failure"].copy()).float()

    # compute the DDQN target
    with torch.no_grad():

        # action selection: online network
        q_online_next = online_q(next_states)
        best_actions = q_online_next.argmax(dim=1)

        # action evaluation: target network
        q_target_next = target_q(next_states)
        q_target_selected = q_target_next.gather(
            1,
            best_actions.unsqueeze(1),
        ).squeeze(1)

        target = rewards + gamma * q_target_selected * (1.0 - failures)

    # compute the current Q(s, a) with the online network
    q_current_all = online_q(states)
    q_current = q_current_all.gather(1, actions.unsqueeze(1)).squeeze(1)

    # compute the mean squared TD error
    td_error = target - q_current
    loss = (td_error ** 2).mean()

    # update the online network
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    return loss.item()
```

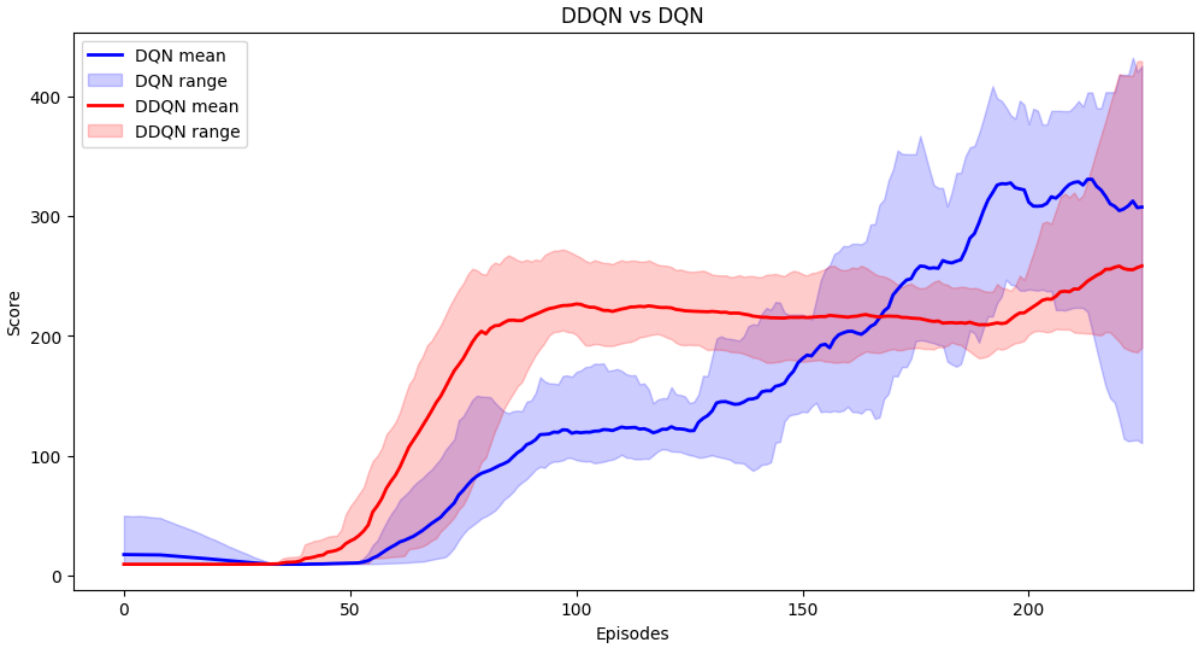
We run the experiment in order to evaluate the impact of this idea:

```
ddqn_results = experiment(
    env=cart_pole,
    train=dqn,
    setup=setup_dqn,
    max_episodes=250,
    name="ddqn",
)
```

```
Experiment seed: 12
Episode 250, score 372.1
Experiment seed: 34
Episode 250, score 500.0
Experiment seed: 56
Episode 250, score 192.8
Experiment seed: 78
Episode 250, score 188.6
Experiment seed: 90
Episode 250, score 297.0
```

To see whether separating action selection from action evaluation improves learning, we compare the DQN and DDQN learning curves across the same set of random seeds:

```
plot_comparison(
    dqn_results,
    ddqn_results,
    first_label="DQN",
    second_label="DDQN",
    first_color="blue",
    second_color="red",
)
```



In this experiment, DQN and DDQN achieve broadly similar performance. DDQN improves earlier and reaches a stable intermediate score more quickly, while DQN catches up later and eventually reaches comparable or slightly higher average scores. The shaded regions also show that both methods remain sensitive to the random seed. This result is not surprising. Cart-Pole is a small environment with only two actions and relatively smooth rewards, so **overestimation bias is not the main source of difficulty**. DDQN is designed to reduce that bias, but its advantage is usually more visible in environments with many actions and noisy value estimates. Since DDQN requires only a small change to the target computation and adds essentially no extra computational cost, it is often preferred in practice even when the improvement is modest in simple environments.

1.3.8 Prioritized Experience Replay (PER)

So far, transitions have been sampled uniformly from the replay buffer. This is simple and useful, because it breaks temporal correlations between consecutive experiences. However, uniform sampling also treats all transitions as equally informative. This is not always ideal. Some transitions are already well understood by the agent and produce very small updates. Others are **surprising**: the target value is very different from the value currently predicted by the network. These surprising transitions are often more useful for learning. The key idea of **Prioritized Experience Replay** (**PER**) is to sample important transitions more often. A natural measure of surprise is the absolute TD error:

$$|\delta| = |r + \gamma Q_{\theta_{\text{target}}}(s', \arg\max_{a'} Q_{\theta_{\text{online}}}(s', a')) - Q_{\theta_{\text{online}}}(s, a)|$$

A large TD error means that the prediction was far from the target, so replaying that transition may produce a useful update. PER therefore assigns each transition a priority based on its TD error:

$$p_i = |\delta_i| + \epsilon$$

The small constant ϵ ensures that even transitions with very small TD error still have a chance of being sampled. To implement this, we extend the experience type with two additional fields: one for the priority and one for the importance-sampling weight used later in the loss:

```
experience_type = np.dtype([
    ("state", np.float32, state_size),
    ("action", np.int8),
    ("reward", np.float32),
    ("next_state", np.float32, state_size),
    ("failure", np.int8),
    ("priority", np.float32),
    ("weight", np.float32),
])
```

New transitions are inserted with the highest priority currently present in the replay memory. This ensures that each new experience is likely to be sampled at least once, before the algorithm has had a chance to estimate its TD error accurately.

```
def store_experience(experience):

    # new transitions receive the highest priority seen so far
    priority = replay_memory.get("max_priority", 1.0)
    weight = 1.0

    replay_memory["buffer"][replay_memory["index"]] = experience + (priority, weight)

    replay_memory["entries"] = min(
        replay_memory["entries"] + 1,
        replay_memory["size"],
    )

    replay_memory["index"] = (replay_memory["index"] + 1) % replay_memory["size"]
```

When sampling a mini-batch, priorities are converted into sampling probabilities:

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}$$

The hyperparameter α controls how strongly priorities affect sampling. When it is 0, all transitions are sampled uniformly. When it is 1, sampling is fully proportional to the priorities. Intermediate values create a compromise between uniform replay and greedy prioritization. This makes sampling stochastic: **higher-priority transitions are more likely to be selected, but lower-priority transitions can still be replayed**.

Prioritized sampling improves learning efficiency, but it also introduces bias. Since **high-error transitions are over-represented**, the gradient computed from a mini-batch is no longer an unbiased estimate of the gradient we would obtain under uniform sampling. To compensate for this, PER uses importance-sampling weights. Transitions that are sampled more often receive smaller weights in the loss:

$$w_i = \left(\frac{1}{N}\frac{1}{P(i)}\right)^\beta$$

Here, N is the number of transitions in the replay memory, and β controls how much bias correction is applied. When it is 0, no correction is used. When it is 1, the correction is fully applied. In practice, its value is often increased gradually during training. The weights are normalized so that the largest weight in the sampled mini-batch is equal to one:

$$w_i \leftarrow \frac{w_i}{\max_j w_j}$$

This keeps the scale of the loss under control.

```
def sample_experiences(
    batch_size=32,
    alpha=0.5,
    beta=0.4,
    priority_epsilon=1e-6,
):

    entries = replay_memory["entries"]

    # convert priorities into sampling probabilities
    priorities = replay_memory["buffer"]["priority"][:entries] + priority_epsilon
    scaled_priorities = priorities ** alpha
    probabilities = scaled_priorities / scaled_priorities.sum()

    # sample transitions according to their probabilities
    indices = np.random.choice(
        entries,
```

```
        batch_size,
        replace=False,
        p=probabilities,
    )

    # compute and normalize importance-sampling weights
    weights = (entries * probabilities[indices]) ** (-beta)
    weights = weights / weights.max()

    replay_memory["buffer"]["weight"][indices] = weights

    return replay_memory["buffer"][indices], indices
```

After the optimization step, we update the priorities of the sampled transitions using their latest TD errors. We update only the sampled transitions, not the entire replay memory, because recomputing TD errors for every stored transition after each gradient step would be too expensive:

```
def update_experiences(indices, td_errors):

    # update priorities of the sampled transitions
    priorities = np.abs(td_errors) + epsilon
    replay_memory["buffer"]["priority"][indices] = priorities

    # keep track of the largest priority for new transitions
    replay_memory["max_priority"] = max(
        replay_memory.get("max_priority", 1.0),
        float(priorities.max()),
    )
```

We can now write the loss used with prioritized replay. The first change is that transitions are no longer sampled uniformly from the replay buffer. They are sampled according to the priority-based distribution:

$P(\mathcal{D})$

The second change is that each squared TD error is multiplied by its importance-sampling weight. This compensates, at least partially, for the bias introduced by prioritized sampling: transitions sampled more often receive smaller weights in the loss. Using the DDQN target, the loss becomes:

$$\mathcal{L}(\theta) = \mathbb{E}_{i \sim P(\mathcal{D})}\Big[w_i\big(r_i + \gamma\, Q_{\theta_{\text{target}}}(s'_i, \arg\max_{a'} Q_{\theta_{\text{online}}}(s'_i, a')) - Q_{\theta_{\text{online}}}(s_i, a_i)\big)^2\Big]$$

We can implement this loss by modifying the optimization step.

```
def optimize(gamma=0.99, batch_size=32, alpha=0.5, beta=0.4, priority_epsilon=1e-6):

    # sample a prioritized mini-batch from the replay memory
    batch, indices = sample_experiences(
        batch_size=batch_size,
        alpha=alpha,
        beta=beta,
        priority_epsilon=priority_epsilon,
    )

    # convert experience fields to tensors
    states      = torch.from_numpy(batch["state"].copy()).float()
    actions     = torch.from_numpy(batch["action"].copy()).long()
    rewards     = torch.from_numpy(batch["reward"].copy()).float()
    next_states = torch.from_numpy(batch["next_state"].copy()).float()
    failures    = torch.from_numpy(batch["failure"].copy()).float()
    weights     = torch.from_numpy(batch["weight"].copy()).float()

    # compute the DDQN target
    with torch.no_grad():

        # action selection: online network
        q_online_next = online_q(next_states)
        best_actions = q_online_next.argmax(dim=1)

        # action evaluation: target network
        q_target_next = target_q(next_states)
        q_target_selected = q_target_next.gather(
            1,
            best_actions.unsqueeze(1),
        ).squeeze(1)

        target = rewards + gamma * q_target_selected * (1.0 - failures)

    # compute the current Q(s, a) with the online network
    q_current_all = online_q(states)
    q_current = q_current_all.gather(1, actions.unsqueeze(1)).squeeze(1)

    # compute TD errors
    td_errors = target - q_current
```

```
    # weight each squared TD error by its importance-sampling weight
    loss = (weights * td_errors.pow(2)).mean()

    # update the online network
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    # update priorities using the latest TD errors
    update_experiences(
        indices,
        td_errors.detach().cpu().numpy(),
    )

    return loss.item()
```

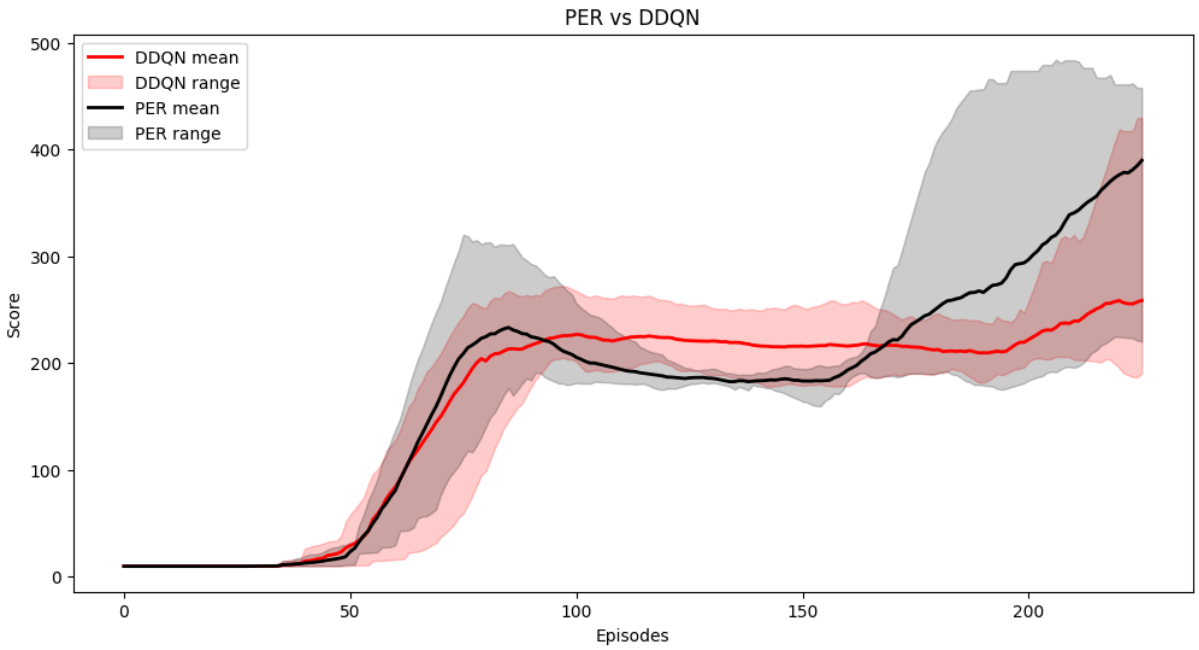
We run the experiment in order to evaluate the impact of this idea:

```
per_results = experiment(
    env=cart_pole,
    train=dqn,
    setup=setup_dqn,
    max_episodes=250,
    name="per",
)
```

```
Experiment seed: 12
Episode 250, score 500.0
Experiment seed: 34
Episode 250, score 217.6
Experiment seed: 56
Episode 250, score 500.0
Experiment seed: 78
Episode 250, score 500.0
Experiment seed: 90
Episode 250, score 500.0
```

After the esperiment, we save results for later use and we make similar statistical operations to create a graph of performance:

```
plot_comparison(
    ddqn_results,
    per_results,
    first_label="DDQN",
    second_label="PER",
    first_color="red",
    second_color="black",
)
plt.show();
```



On Cart-Pole, the benefit of PER is present but not dramatic. The PER curve follows a pattern similar to DDQN in the early part of training, then improves more sharply near the end of the run. This suggests that prioritized sampling can help the agent focus on more informative transitions, but the advantage is not immediate or uniformly stable. This modest gain is expected. Cart-Pole is a small environment with only two actions, and its TD errors are usually not as diverse as in larger, noisier domains. As a result, prioritizing transitions does not radically change the information available to the network. PER also adds computational cost. In our simple implementation, the sampling probabilities are recomputed over the whole replay memory at every optimization step, which costs $O(N)$ in the size of the memory.

1.3.9 Dueling Architecture

The improvements introduced so far have mainly made deep Q-learning look more like supervised learning. Replay memory makes the data less correlated, and the target network makes the labels more stable. These changes are crucial, but they do not change the way the Q-network represents values. The **dueling architecture** is different: it modifies the network itself. Its goal is to exploit a useful structure in the way the value functions relate to one another. The state-value function measures how good it is to be in a state:

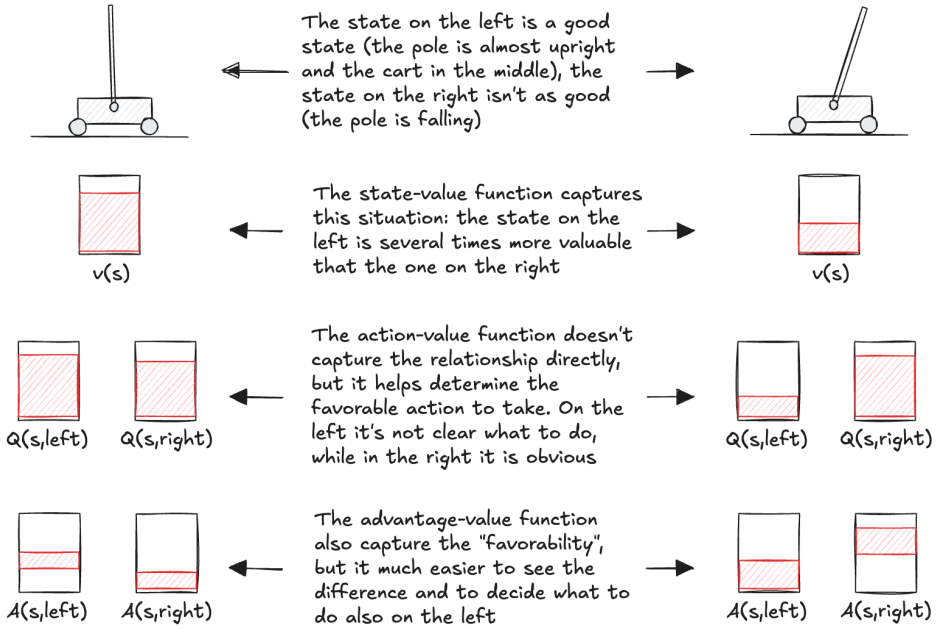
$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$

The action-value function measures how good it is to take a specific action in that state:

$q_\pi(s,a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$

We can introduce the **advantage function** to measure how much better or worse an action is compared with the value of the state itself:

$a_\pi(s,a) = q_\pi(s,a) - v_\pi(s)$

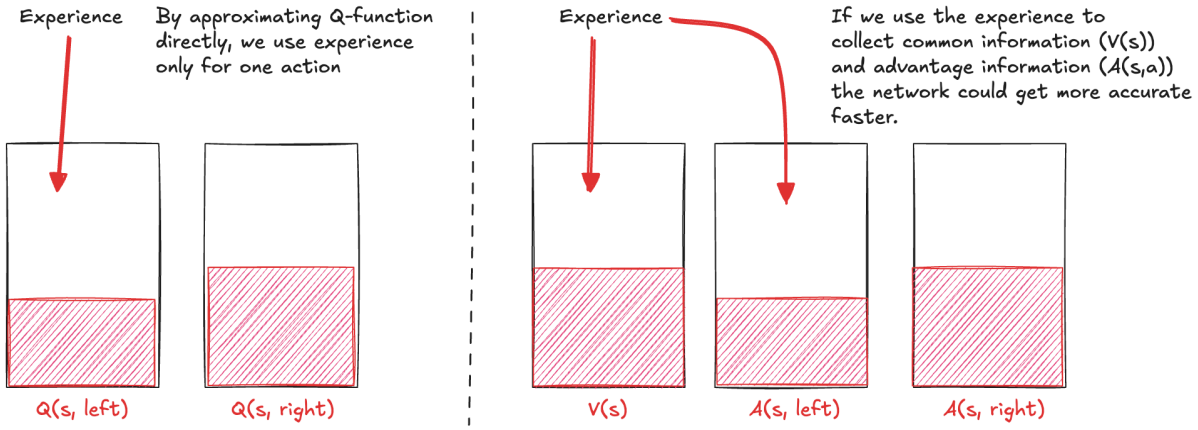


Equivalently, the action-value function can be decomposed as:

$q_\pi(s,a) = v_\pi(s) + a_\pi(s,a)$

Intuitively, the state-value function captures what is common to all actions in a state, while the advantage function captures the relative preference among actions. This distinction is useful because, in many states, the value of the state is much easier to estimate than the exact value of every action. For example, in Cart-Pole, a state where the pole is almost upright is generally

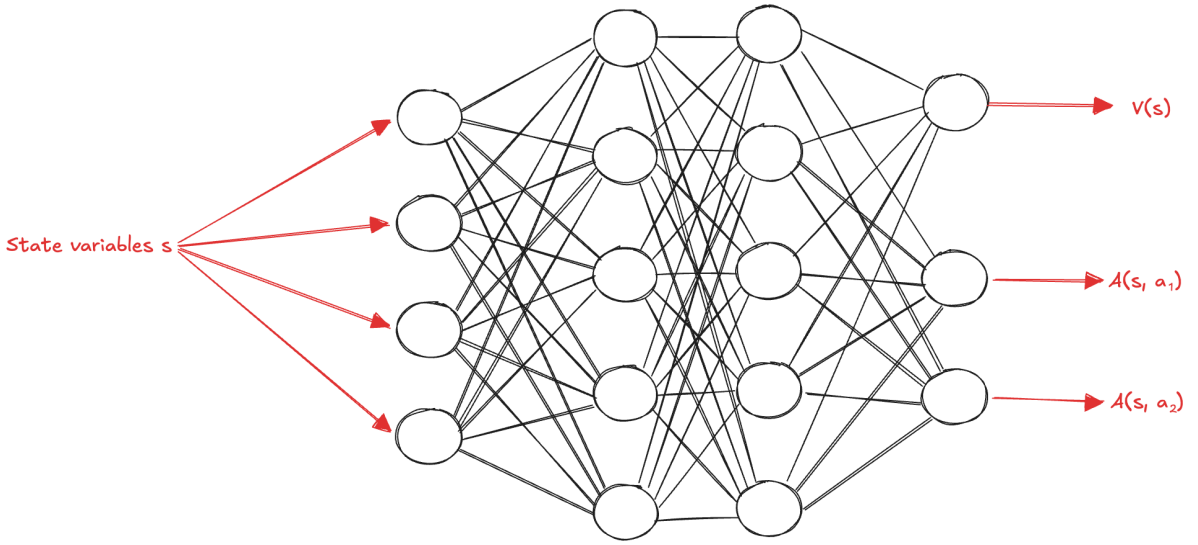
good, regardless of whether the best action is slightly left or slightly right. Conversely, a state where the pole is already falling is less valuable overall, even if one action is still better than the other. A standard Q-network must learn a separate value for each action directly. This can be inefficient, because part of what it learns is shared across actions. The dueling architecture makes this shared structure explicit:



The [**Dueling Network Architecture**] (./papers/2016%20-%20Dueling%20Network%20Architectures%20for%
uses one shared feature extractor followed by two separate heads:

- a **value head**, which estimates the value of the state
- an **advantage head**, which estimates one advantage value for each action

The value head outputs a single number. The advantage head outputs a vector with one entry per action.



The two heads must then be combined to reconstruct the Q-values. A first idea would be to

simply add them:

$$Q(s,a;\theta,\theta_V,\theta_A) = V(s;\theta,\theta_V) + A(s,a;\theta,\theta_A)$$

However, this decomposition is **not identifiable**. The training signal only constrains the sum. We could add a constant to the value estimate and subtract the same constant from all advantages, and the resulting Q-values would remain unchanged. As a result, nothing forces the value head to learn a true state value, or the advantage head to learn meaningful relative advantages. To remove this ambiguity, the advantages are centered before being added to the value. In practice, the dueling architecture subtracts the mean advantage:

$$Q(s,a;\theta,\theta_V,\theta_A) = V(s;\theta,\theta_V) + \left(A(s,a;\theta,\theta_A) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s,a';\theta,\theta_A) \right)$$

With this aggregation, the advantages have zero mean across actions. The value stream captures what is common to all actions in the state, while the advantage stream captures only the relative differences between actions. We implement this architecture by defining a custom PyTorch module. A simple "Sequential" model is no longer enough, because the network must split into two heads and then recombine their outputs with the centering operation.

```
class DuelingNetwork(torch.nn.Module):

    def __init__(self):
        super().__init__()

        # shared feature extractor
        self.shared = torch.nn.Sequential(
            torch.nn.Linear(state_size[0], first_hidden_layer),
            torch.nn.ReLU(),
            torch.nn.Linear(first_hidden_layer, second_hidden_layer),
            torch.nn.ReLU(),
        )

        # value stream: one value per state
        self.value_head = torch.nn.Linear(second_hidden_layer, 1)

        # advantage stream: one advantage per action
        self.advantage_head = torch.nn.Linear(second_hidden_layer, action_size)

    def forward(self, state):

        features = self.shared(state)

        value = self.value_head(features)
```

```
        advantages = self.advantage_head(features)

        # works for both one state and a batch of states
        advantages = advantages - advantages.mean(dim=-1, keepdim=True)

        return value + advantages
```

```
def create_network():
    dnn = DuelingNetwork();
    return dnn;
```

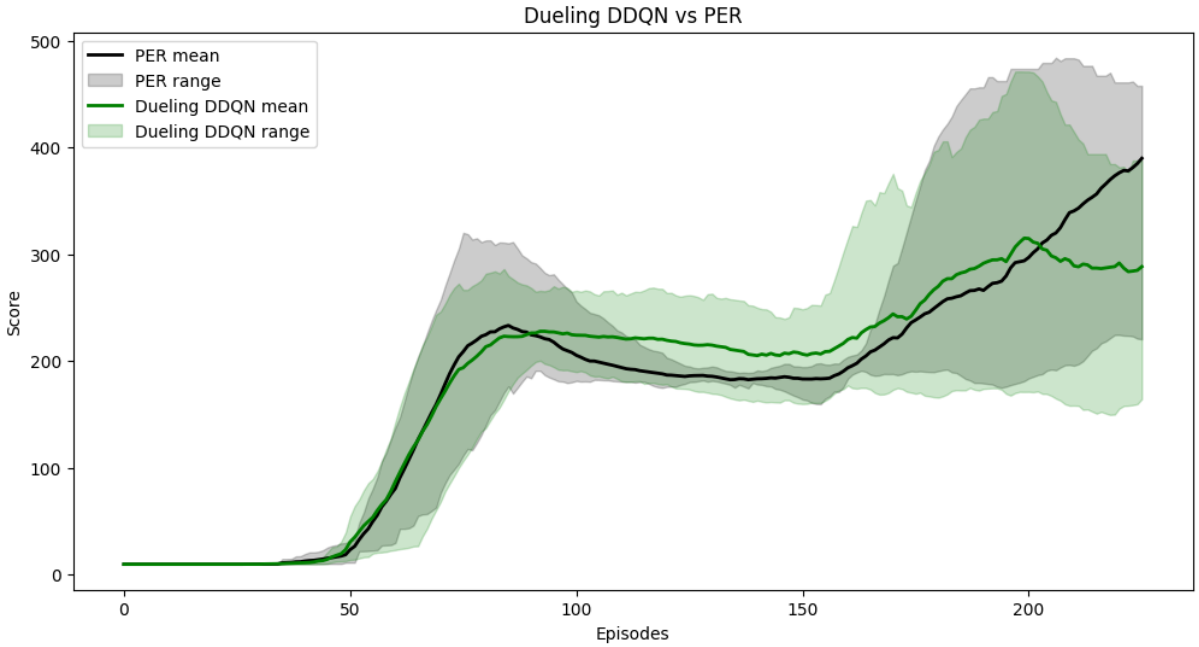
The rest of the algorithm remains the same as DDQN. We can now run the same experiment with the dueling architecture. The training loop is unchanged; only the network architecture used for the online and target networks is different:

```
dueling_results = experiment(
    env=cart_pole,
    train=dqn,
    setup=setup_dqn,
    max_episodes=250,
    name="dueling",
)
```

```
Experiment seed: 12
Episode 250, score 500.0
Experiment seed: 34
Episode 250, score 498.6
Experiment seed: 56
Episode 250, score 260.9
Experiment seed: 78
Episode 250, score 195.4
Experiment seed: 90
Episode 250, score 500.0
```

```
plot_comparison(
    per_results,
    dueling_results,
    first_label="PER",
```

```
    second_label="Dueling DDQN",
    first_color="black",
    second_color="green",
)
```



In Cart-Pole, Dueling DDQN performs similarly to the previous agents. Its learning curve follows PER closely for much of training, with only modest differences in the final scores and a still sizeable range across seeds. This is expected. Cart-Pole has only two actions, so there is limited benefit in separating the value of the state from the relative advantage of each action. The shared state-value estimate helps most when many actions have similar effects, or when the value of being in a state is easier to learn than the value of each individual action. The dueling architecture is therefore more useful in environments with larger action spaces. In those settings, many actions may have little or no effect in a given state, and the network benefits from learning the state value once instead of relearning it separately for every action.

1.4 Limitations and Outlook

The improvements introduced in this chapter can be combined in several ways to obtain more stable and efficient agents. A well-known example is **Rainbow**, which combines DDQN, prioritized replay, dueling networks, and three additional ideas: multi-step targets, distributional value learning, and noisy networks for exploration. Even with these refinements, important limitations remain.

First, value-based deep reinforcement learning is **highly sensitive to hyperparameters**. Small changes in the learning rate, exploration schedule, replay buffer size, target update frequency, or network architecture can have large effects on performance. In practice, many reasonable-looking configurations fail, and only a narrow range works well.

Second, these methods rely heavily on the assumption that the state is **Markovian**: the current observation must contain all the information needed to make good decisions. This matters especially for bootstrapping, because a bootstrapped target is only as reliable as the value estimate of the next state. If the state is incomplete or partially observable, the target can become misleading. Methods that bootstrap less, such as Monte Carlo methods, and the policy-based methods introduced in the next chapters, are often less sensitive to this issue.

Finally, DQN and its variants still contain the ingredients of the **deadly triad**: function approximation, bootstrapping, and off-policy learning. Each ingredient can be useful on its own, but their combination can make value estimates unstable or even divergent. Replay buffers, target networks, double learning, conservative learning rates, and careful tuning reduce these risks, but they do not remove them entirely.

For this reason, value-based deep reinforcement learning is powerful but delicate. It works best when the algorithmic refinements are used together with careful experimentation, diagnostics, and patience.